TEAM NOTEBOOK

NAMES ARE IRRELEVANT

SUST

Contents

1 DP **2**
- 1.1 ConvexHullTrick . . . 2
- 1.2 Digit DP optimize(1 memset) . . . 2
- 1.3 Digit DP . . . 2
- 1.4 Divide And Conquere . . . 3
- 1.5 KnuthOptimization . . . 3
- 1.6 LIS . . . 3
- 1.7 MCM . . . 3
- 1.8 SOS DP . . . 4

2 Data structure **4**
- 2.1 2d$_{sparse_{T}able}$ . . . 4
- 2.2 BIT range update and query . . . 4
- 2.3 BIT . . . 5
- 2.4 DSU . . . 5
- 2.5 Dynamic Lower bound . . . 5
- 2.6 Mos . . . 5
- 2.7 SCC . . . 6
- 2.8 SegmentTree . . . 6
- 2.9 Sparse table . . . 7
- 2.10 SquareRoot . . . 7
- 2.11 TernarySearch . . . 7
- 2.12 next$_{smallerprevious smaller}$ . . . 7
- 2.13 ordered set . . . 7
- 2.14 segment tree persistent lazy . . . 8
- 2.15 segment tree point update and max subarray sum . . . 8
- 2.16 wavelet tree . . . 9
- 2.17 xor basis . . . 9

3 Geo **10**
- 3.1 Geo . . . 10
- 3.2 Grundy . . . 16
- 3.3 build . . . 16
- 3.4 formula . . . 17
- 3.5 pragma . . . 17
- 3.6 template . . . 17

4 Graph Theory **17**
- 4.1 Articulation bridge . . . 17
- 4.2 Articulation point . . . 17
- 4.3 Bellman ford . . . 17
- 4.4 Dijsktra . . . 18

5 Number theory **18**
- 5.1 bigmod . . . 18
- 5.2 extended euclid . . . 18
- 5.3 linear diphantine equation . . . 18
- 5.4 linear sieve . . . 18
- 5.5 mat expo . . . 18
- 5.6 ncr for mod . . . 19
- 5.7 phi . . . 19
- 5.8 pollard rho . . . 19
- 5.9 power tower . . . 19
- 5.10 sieve all . . . 20
- 5.11 sieve . . . 20
- 5.12 totient . . . 20

6 String **20**
- 6.1 AhoCorasick . . . 20
- 6.2 Hashing . . . 20
- 6.3 LPS . . . 21
- 6.4 Manachar . . . 21
- 6.5 MaximumLexicographicRotation . . . 21
- 6.6 MinimumLexicographicRotaion . . . 21
- 6.7 PersistentBitwiseTrie . . . 21
- 6.8 SuffixArray . . . 21
- 6.9 SuffixAutomation . . . 22
- 6.10 Trie . . . 23
- 6.11 bitTrie . . . 23
- 6.12 kmp . . . 23
- 6.13 z algo . . . 23

7 Tree **23**
- 7.1 Centriod Decomposition . . . 23
- 7.2 Eular tour . . . 24
- 7.3 HLD(update on edge) . . . 24
- 7.4 HLD(update on node) . . . 25
- 7.5 LCA . . . 25

1 DP

1.1 ConvexHullTrick

```
const ll is_query = -(1LL<<62);
struct line {
    ll m, b;
    mutable function<const line*()> succ;
    bool operator<(const line& rhs) const {
        if (rhs.b != is_query) return m < rhs.m;
        const line* s = succ();
        if (!s) return 0;
        ll x = rhs.m;
        return b - s->b < (s->m - m) * x;
    }
};
struct dynamic_hull : public multiset<line> { // will
     maintain upper hull for maximum
    const ll inf = LLONG_MAX;
    bool bad(iterator y) {
        auto z = next(y);
        if (y == begin()) {
            if (z == end()) return 0;
            return y->m == z->m && y->b <= z->b;
        }
        auto x = prev(y);
        if (z == end()) return y->m == x->m && y->b <= x->b;
  /* compare two lines by slope, make sure denominator is
       not 0 */
        ll v1 = (x->b - y->b);
        if (y->m == x->m) v1 = x->b > y->b ? inf : -inf;
        else v1 /= (y->m - x->m);
        ll v2 = (y->b - z->b);
        if (z->m == y->m) v2 = y->b > z->b ? inf : -inf;
        else v2 /= (z->m - y->m);
        return v1 >= v2;
    }
    void insert_line(ll m, ll b) {
        auto y = insert({ m, b });
        y->succ = [=] { return next(y) == end() ? 0 : &*next(
             y); };
        if (bad(y)) { erase(y); return; }
        while (next(y) != end() && bad(next(y))) erase(next(y
             ));
        while (y != begin() && bad(prev(y))) erase(prev(y));
    }
    ll eval(ll x) {
        auto l = *lower_bound((line) { x, is_query });
        return l.m * x + l.b;
    }
};
dynamic_hull cht;
memset(dp, 0, sizeof(dp));
cht.insert_line(0, 0);
dp[i] = p[i] * b[i] - cht.eval(b[i]);
cht.insert_line(p[i - 1], -dp[i]);
```

1.2 Digit DP optimize(1 memset)

```
ll dp[20][1030][2][2];
ll casio[20][1030][2][2];
int cur;
int v[20];
string s;
int n;
ll foo(int pos, int mask, int ok, bool other){
    if (pos == -1){
        int tb = 0;
        int mxdig = -1;
        for (int i = 0 ; i <= 9 ; i++){
            if (mask & (1 << i))
                tb++, mxdig = max(mxdig, i);
        }
        return tb == mxdig;
    }
    ll &R = dp[pos][mask][ok][other];
    if(casio[pos][mask][ok][other]==cur)
        return R;
    casio[pos][mask][ok][other] = cur;
    if(ok && ~R)
        return R;
    int dgt = 9;
    if (!ok){
        dgt = v[pos];
    }
    R = 0;
    for (int i = 0 ; i <= dgt; i++){
        int temp = mask;
        if (other)
            temp |= (1 << i);
        else{
            if (i){
                other = 1;
                temp |= (1 << i);
            }
        }
        if (i < dgt || ok)
            R += foo(pos - 1, temp , true, other);
        else
            R += foo(pos - 1, temp , false, other);
    }
    return R;
}
void pro()
{
 memset()
    ll x;
    cin >> x;
    for(int i=0 ; i<20 ; i++)
    {
        v[i] = x%10;
        x/=10;
    }
    cur++;
    ll ans = foo(18, 0, 0, 0);
    cout << ans << endl;
}
```

1.3 Digit DP

```
const int N = 20;
int a[N];
ll dp[N][11][2][2]; // digit dp te amar number generate hoy
     emne : 0, 01, 02, 03, 04, 05.....066,0667

ll getsum(int pos, int dig, int n, bool ok, bool other) {
    if (pos > n) {
        return 1;
    }
    ll &R = dp[pos][dig][ok][other];
    if (R != -1)
        return R;
    int maxdigit = 9;
    if (!ok)
        maxdigit = a[pos];
    ll res = 0;
    for (int i = 0 ; i <= maxdigit ; i++) {
        if (dig == i && other)
            continue;
        if (i > 0)
            other = 1;
        if (i < maxdigit || ok)
            res += getsum(pos + 1, i, n, true, other);
        else
            res += getsum(pos + 1, i, n, false, other);
    }
    return R = res;
}
```

```
void Solve() {

    string l, r;
    cin >> l >> r;
    int n = r.size();
    r = '*' + r;
    for (int i = 0 ; i <= n ; i++) {
        a[i] = r[i] - '0';
    }
    memset(dp, -1, sizeof(dp));
    ll sumr = getsum(1, -1 , n, 0, 0);
    n = l.size();
    l = '*' + l;
    for (int i = 0 ; i <= n ; i++)
        a[i] = l[i] - '0';
    memset(dp, -1, sizeof(dp));
    ll suml = getsum(1, -1, n, 0, 0);
    bool ok = 1;
    for (int i = 2 ; i <= n ; i++)
        if (l[i - 1] == l[i])
            ok = 0;
    cout << sumr - suml + ok << endl;
}
```

1.4 Divide And Conquere

```
ll dp[2][4010]; ll mat[4010][4010];
ll cost(ll l1, ll r1, ll l2, ll r2)
{
 ll sum = mat[l2][r2];
 sum -= (mat[l2][r1 - 1] + mat[l1 - 1][r2]);
 sum += mat[l1 - 1][r1 - 1];
 return sum / 2;
}
ll cnt = 0;
void compute(int groupNo, int L, int R, int OptL, int OptR)
{
 if (L > R)
  return;
 int mid = (L + R) / 2;
 dp[(groupNo & 1)][mid] = inf;
 int optNow = OptL;
 for (int endOfLast = OptL; endOfLast <= OptR && endOfLast <
       mid; endOfLast++)
 {
  ll ret = cost(endOfLast + 1, endOfLast + 1, mid, mid);
  if (dp[(groupNo & 1) ^ 1][endOfLast] == -1)
   ret += inf;
  else ret += dp[(groupNo & 1) ^ 1][endOfLast];
  if (ret < dp[(groupNo & 1)][mid])
  {
   dp[(groupNo & 1)][mid] = ret; optNow = endOfLast;
  }
 }
 compute(groupNo, L, mid - 1, OptL, optNow);
 compute(groupNo, mid + 1, R, optNow, OptR);
}
void solve()
{
 ll N, K; cin >> N >> K;
 for (int i = 1; i <= N; i++)
 {
  for (int j = 1; j <= N; j++)
   cin >> mat[i][j];
 }
 for (int i = 1; i <= N; i++)
 {
  for (int j = 1; j <= N; j++)
   mat[i][j] += mat[i][j - 1];
  for (int j = 1; j <= N; j++)
   mat[i][j] += mat[i - 1][j];
 }
 memset(dp, -1, sizeof(dp));
 dp[0][0] = 0;
 for (int groupNo = 1; groupNo <= K; groupNo++)
  compute(groupNo, 1, N, 0, N);
 cout << dp[K & 1][N] << endl;
}
```

1.5 KnuthOptimization

```
for (int pos = 1; pos <= N; pos++)
{
 Opt[0][pos] = 0;
}
for (int groupNo = 1; groupNo <= K; groupNo++)
{
 Opt[groupNo][N + 1] = N;
}
for (int groupNo = 1; groupNo <= K; groupNo++)
{
 for (int pos = N; pos >= 1; pos--)
 {
  for (int endOfLast = Opt[groupNo - 1][pos];
         endOfLast <= Opt[groupNo][pos + 1]; endOfLast++)
  {
   int ret = dp[groupNo - 1][endOfLast] + Cost(endOfLast +
        1, pos);
   if (dp[groupNo][pos] <= ret)
    continue;
   dp[groupNo][pos] = ret;
   Opt[groupNo][pos] = endOfLast;
  }
 }
}
```

1.6 LIS

```
vector<int> LIS(vector<int>& nums) {
    int n = nums.size();
    vector<int> lis;
    vector<int> par(n, -1);
    vector<int> indices;
    vector<int> pos(n, 0);
    for (int i = 0; i < n; ++i) {
        auto it = lower_bound(lis.begin(), lis.end(), nums[i
            ]);
        int idx = it - lis.begin();
        if (it == lis.end()) {
            lis.push_back(nums[i]);
        } else {
            *it = nums[i];
        }
        pos[idx] = i;
        if (idx > 0) {
            par[i] = pos[idx - 1];
        }
    }
    vector<int> res;
    int cur = pos[lis.size() - 1];
    while (cur != -1) {
        res.push_back(nums[cur]);
        cur = par[cur];
    }
    reverse(res.begin(), res.end());
    return res;
}
```

1.7 MCM

```
const int INF = 1e9;
// Memoization table to store the results of subproblems
vector<vector<int>> dp;
// Recursive function to find the minimum number of
     multiplications needed
```

```
int mcm(vector<int>& dims, int i, int j) {
    if (i == j) {
        return 0; // Only one matrix, no multiplications
             needed
    }
    if (dp[i][j] != -1) {
        return dp[i][j]; // Return already computed result
    }
    int minCost = INF;
    // Try every possible position to split the chain
    for (int k = i; k < j; k++) {
        int cost = mcm(dims, i, k) + mcm(dims, k + 1, j) +
             dims[i - 1] * dims[k] * dims[j];
        minCost = min(minCost, cost);
    }
    dp[i][j] = minCost; // Store the result in dp table
    return dp[i][j];
}
int main() {
    vector<int> dims = {40, 20, 30, 10, 30}; // Example
         dimensions of matrices
    int n = dims.size() - 1;
    // Initialize dp table with -1 (indicating uncomputed
         values)
    dp.assign(n + 1, vector<int>(n + 1, -1));
    int result = mcm(dims, 1, n);
    cout << "Minimum number of multiplications is " << result
         << endl;
    return 0;
}
```

1.8 SOS DP

```
const ll MLOG = 20; const ll MAXN = (1 << MLOG);
ll dp[sz + 10], fre[sz + 10], mp[sz + 10];
// forward1: Propagates values from subsets to their
     supersets
void forward1() {
  for (int bit = 0; bit < MLOG; ++bit) {
    for (int i = 0; i < MAXN; ++i) {
      if (i & (1 << bit)) {
        dp[i] += dp[i ^ (1 << bit)];
      }
    }
  }
}
// backward1: Reverses the effect of forward1 by removing
     contributions from supersets
// This is used when dp[i] contains info about all subsets
     of i,
// and we want to isolate the info for only i.
void backward1() {
  for (int bit = 0; bit < MLOG; ++bit) {
    for (int i = MAXN - 1; i >= 0; --i) {
      if (i & (1 << bit)) {
        dp[i] -= dp[i ^ (1 << bit)];
      }
    }
  }
}
// forward2: Propagates values from supersets to their
     subsets
void forward2() {
  for (int bit = 0; bit < MLOG; ++bit) {
    for (int i = MAXN - 1; i >= 0; --i) {
      if (i & (1 << bit)) {
        dp[i ^ (1 << bit)] += dp[i];
      }
    }
  }
}
// backward2: Reverses the effect of forward2 by removing
     contributions from subsets
// This is used when dp[i] contains info about all supersets
     of i,
// and we want to isolate the info for only i.
void backward2() {
  for (int bit = 0; bit < MLOG; ++bit) {
    for (int i = 0; i < MAXN; ++i) {
      if (i & (1 << bit)) {
        dp[i ^ (1 << bit)] -= dp[i];
      }
    }
  }
}
void print(vector<ll> &v) {
  for (int i = 0; i < v.size(); i++) {
    cout << v[i] << (i == v.size() - 1 ? "\n" : " ");
  }
}
void solve() {
  ll n; cin >> n; vector<ll> v(n);
  memset(dp, 0, sizeof(dp));
  memset(fre, 0, sizeof(fre));
  memset(mp, 0, sizeof(mp));
  for (int i = 0; i < n; i++) {
    cin >> v[i]; dp[v[i]]++;
    fre[v[i]]++; mp[v[i]] = 1;
  }
  forward1();
  ll sum = 0;
  for (int i = 0; i < sz; i++) {
    if (mp[i]) {
      sum += (dp[i] - fre[i]) * fre[i];
      sum += (fre[i] * (fre[i] - 1)) / 2;
    }
  }
  cout << sum << endl;
}
```

2 Data structure

2.1 2d$_{Sparse_Table}$

```
// Build Sparse Table
void buildSparseTable(vector<vector<int>>& mat) {
    int row = mat.size(), col = mat[0].size();
    for (int j = 0; j <= LOG; j++) for (int u = 0; u < row; u
         ++) for (int v = 0; v < col; v++)
            sp[u][v][j][0] = (j == 0) ? mat[u][v] : (v +
                 (1 << (j - 1)) < col ? min(sp[u][v][j -
                 1][0], sp[u][v + (1 << (j - 1))][j -
                 1][0]) : sp[u][v][j - 1][0]);
    for (int i = 1; i <= LOG; i++) for (int j = 0; j <= LOG;
         j++) for (int u = 0; u < row; u++) for (int v = 0; v
          < col; v++)
                sp[u][v][j][i] = (u + (1 << (i - 1)) < row
                     ) ? min(sp[u][v][j][i - 1], sp[u + (1
                      << (i - 1))][v][j][i - 1]) : sp[u][v
                     ][j][i - 1];
}
// Query Sparse Table
ll querySparse(ll l, ll r, ll width, ll height) {
    ll res = 1e9;
    for (int i = 0; i <= LOG; i++) if ((height >> i) & 1) for
          (ll tl = l, tr = r, j = 0; j <= LOG; j++)
            if ((width >> j) & 1) res = min(res, sp[tl][tr
                 ][j][i]), tr += (1 << j); l += (1 << i);
    return res;
}
```

2.2 BIT range update and query

```
const int N = 3e5 + 9;
```

```
struct BIT {
  long long M[N], A[N];
  BIT() {
    memset(M, 0, sizeof M); memset(A, 0, sizeof A);
  }
  void update(int i, long long mul, long long add) {
    while (i < N) {
      M[i] += mul; A[i] += add; i |= (i + 1);
    }
  }
  void upd(int l, int r, long long x) {
    update(l, x, -x * (l - 1)); update(r, -x, x * r);
  }
  long long query(int i) {
    long long mul = 0, add = 0; int st = i;
    while (i >= 0) {
      mul += M[i]; add += A[i];
      i = (i & (i + 1)) - 1;
    }
    return (mul * st + add);
  }
  long long query(int l, int r) {
    return query(r) - query(l - 1);
  }
} t;
```

2.3 BIT

```
struct BIT { vector<int> bit; int size; BIT(int n) { size =
     n; bit.resize(size + 1, 0); }
void update(int id, int val) { id++; while (id <= size) {
     bit[id] += val; id = id + (id & -id); } }
int sum(int id) { int sum = 0; while (id > 0) { sum += bit[
     id]; id = id - (id & -id); } return sum; }
int rangeSum(int l, int r) { return sum(r + 1) - sum(l); }
     };
```

2.4 DSU

```
struct DSU {
    vector<ll> parent, Rank;
    DSU(ll n) { parent.resize(n + 1), Rank.resize(n + 1, 1);
         for (ll i = 1; i <= n; i++) parent[i] = i; }
    ll find_parent(ll v) { return v == parent[v] ? v : parent
         [v] = find_parent(parent[v]); }
    bool same_set(ll a, ll b) { return find_parent(a) ==
         find_parent(b); }
    void union_sets(ll a, ll b) { a = find_parent(a), b =
         find_parent(b); if (a != b) { if (Rank[a] > Rank[b])
          swap(a, b); parent[a] = b, Rank[b] += Rank[a]; } }
};
```

2.5 Dynamic Lower bound

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <limits>
using namespace std;
const int inf = 1e9;
struct MergeSortTree
{
    int n; vector<vector<int>> tree;
    MergeSortTree(const vector<int>& arr)
    {
        n = arr.size(); tree.resize(4 * n); build(0, 0, n -
             1, arr);
    }
    void build(int node, int start, int end, const vector<int
         >& arr)
    {
        if (start == end)
        {
            tree[node].push_back(arr[start]); return;
        }
        int mid = (start + end) / 2;
        build(2 * node + 1, start, mid, arr);
        build(2 * node + 2, mid + 1, end, arr);
        merge(tree[2 * node + 1].begin(), tree[2 * node + 1].
             end(),
              tree[2 * node + 2].begin(), tree[2 * node + 2].
                  end(),
              back_inserter(tree[node]));
    }
    // lower bound of x in the range [l, r]
    int query(int node, int start, int end, int left, int
         right, int x)
    {
        if (left > end || right < start)
            return inf;
        if (left <= start && right >= end) {
            auto itr = lower_bound(tree[node].begin(), tree[
                 node].end(), x);
            if (itr != tree[node].end())
                return *itr;
            return inf;
        }
        int mid = (start + end) / 2;
        int leftLowerBound = query(2 * node + 1, start, mid,
             left, right, x);
        int rightLowerBound = query(2 * node + 2, mid + 1,
             end, left, right, x);
        return min(leftLowerBound, rightLowerBound);
    }
    // Wrapper function for querying the tree
    int query(int left, int right, int x)
    {
        return query(0, 0, n - 1, left, right, x);
    }
};

int main() {
    vector<int> arr = {3, 1, 5, 2, 4};
    MergeSortTree mst(arr);
    cout << mst.query(1, 2, 2) << endl; // Output: 2 (index 2
          is the lower bound of 2 in the range [1, 3])
    cout << mst.query(0, 4, 0) << endl; // Output: 3 (index 3
          is the lower bound of 3 in the range [0, 4])
    cout << mst.query(0, 4, 6) << endl; // Output: inf (6 is
         not present in the range [0, 4])
    return 0;
}
```

2.6 Mos

```
struct Query {
    ll id, l, r, bno;
};
// basic comparator
bool cmp(Query &a, Query &b) {
    if (a.bno != b.bno) {
        return a.bno < b.bno;
    } else if (a.bno & 1) {
        return a.r < b.r;
    } else {
        return a.r > b.r;
    }
}
ll n, q; ll bs = 700; // change is depending on problem
Query qry[N]; ll a[N]; ll ans[N];
ll lp, rp, cnt; ll freq[1000001];
void add(ll val) {
    freq[val]++;
    if (freq[val] == 1) cnt++;
}
```

```
void remove(ll val) {
    freq[val]--;
    if (freq[val] == 0) cnt--;
}
ll get_answer() {
    return cnt;
}
void solve() {
    cin >> n;
    for (ll i = 0; i < n; i++) {
        cin >> a[i];
    }
    cin >> q;
    for (ll i = 0; i < q; i++) {
        ll l, r; cin >> l >> r; --l; --r;
        qry[i].id = i; qry[i].l = l;
        qry[i].r = r; qry[i].bno = l / bs;
    }
    sort(qry, qry + q, cmp);
    lp = 0; rp = 0; cnt = 0; add(a[0]);
    for (ll i = 0; i < q; i++) {
        ll l = qry[i].l; ll r = qry[i].r;
        while (rp < r) {
            add(a[++rp]);
        }
        while (lp > l) {
            add(a[--lp]);
        }
        while (rp > r) {
            remove(a[rp--]);
        }
        while (lp < l) {
            remove(a[lp++]);
        }
        ans[qry[i].id] = get_answer();
    }
    for (ll i = 0; i < q; i++) {
        cout << ans[i] << endl;
    }
}

// this requires the element to be added on both side l, r
    and removed l, r
// but we can also use mo's when data structure allows
    removal from the side where we added, like rollback DSU
```

2.7 SCC

```
struct scc { vector<vector<ll>> edges, redges; vector<bool>
    used; vector<ll> order, component; ll n; void init(ll
    _n) { n = _n; edges = vector<vector<ll>>(n); redges =
    vector<vector<ll>>(n); order.clear(); component.clear()
    ; } void edge(ll u, ll v) { edges[u].push_back(v);
    redges[v].push_back(u); } void dfs1(ll v) { used[v] =
    true; for (ll i = 0; i < edges[v].size(); i++) { if (!
    used[edges[v][i]]) { dfs1(edges[v][i]); } } order.
    push_back(v); } void dfs2(ll v) { used[v] = true;
    component.push_back(v); for (ll i = 0; i < redges[v].
    size(); i++) { if (!used[redges[v][i]]) { dfs2(redges[v
    ][i]); } } } vector<vector<ll>> run() { vector<vector<
    ll>> components; used.assign(n, false); for (ll i = 0;
    i < n; i++) { if (!used[i]) { dfs1(i); } } used.assign(
    n, false); for (ll i = 0; i < n; i++) { ll v = order[n
    - 1 - i]; if (!used[v]) { dfs2(v); components.push_back
    (component); component.clear(); } } return components;
    } };
```

2.8 SegmentTree

```
//Lazy
struct SegmentTree
{
    vector<ll> seg, lazy, v;
    ll n;
    SegmentTree(ll n, vector<ll> v)
    {
        this->n = n; this->v = v;
        seg.resize(4 * n); lazy.resize(4 * n); build(0, 0, n
            - 1);
    }
    void build(ll id , ll low, ll high)
    {
        if (low == high)
        {
            seg[id] = v[low]; return;
        }
        ll mid = (low + high) / 2;
        build(2 * id + 1, low, mid); build(2 * id + 2, mid +
            1, high);
        seg[id] = seg[2 * id + 1] + seg[2 * id + 2];
    }
    void applyLazyUpdates(ll id, ll low, ll high)
    {
        if (lazy[id] != 0)
        {
            seg[id] += (high - low + 1) * lazy[id];
            if (low != high)
            {
                lazy[2 * id + 1] += lazy[id]; lazy[2 * id + 2]
                    += lazy[id];
            }
            lazy[id] = 0;
        }
    }
    void update(ll id, ll low , ll high, ll l , ll r, ll val)
    {
        applyLazyUpdates(id, low, high);
        if (high < l or r < low)
            return;
        if (l <= low and high <= r)
        {
            seg[id] += (high - low + 1) * val;
            if (low != high)
            {
                lazy[2 * id + 1] += val; lazy[2 * id + 2] +=
                    val;
            }
            return;
        }
        ll mid = (low + high) / 2;
        update(2 * id + 1, low, mid, l, r, val);
        update(2 * id + 2, mid + 1, high, l, r, val);
        seg[id] = seg[2 * id + 1] + seg[2 * id + 2];
    }
    ll query(ll id, ll low, ll high, ll l , ll r)
    {
        applyLazyUpdates(id, low, high);
        if (high < l or r < low) return 0;
        if (l <= low and high <= r) return seg[id];
        ll mid = (low + high) / 2;
        ll left = query(2 * id + 1, low, mid, l, r);
        ll right = query(2 * id + 2, mid + 1, high, l, r);
        return left + right;
    }
};
// point
struct SegmentTree
{
    vector<ll> seg, v; ll n;
    SegmentTree(ll n, vector<ll> v)
    {
        this->n = n; this->v = v;
        seg.resize(4 * n); build(0, 0, n - 1);
    }
    void build(ll id, ll low, ll high)
    {
        if (low == high)
```

```
        {
            seg[id] = v[low]; return;
        }
        ll mid = (low + high) / 2;
        build(2 * id + 1, low, mid);
        build(2 * id + 2, mid + 1, high);
        seg[id] = min(seg[2 * id + 1], seg[2 * id + 2]);
    }

    ll query(ll id, ll low , ll high, ll l , ll r)
    {
        if (high < l || r < low)
            return inf;
        if (l <= low and high <= r)
            return seg[id];
        ll mid = (low + high) / 2;
        ll x = query(2 * id + 1, low, mid, l, r);
        ll y = query(2 * id + 2, mid + 1, high, l, r);
        return min(x, y);
    }
    void update(ll id , ll low, ll high, ll i, ll val)
    {
        if (low == high)
        {
            seg[id] = val; v[i] = val; return;
        }
        ll mid = (low + high) / 2;
        if (i <= mid)
            update(2 * id + 1, low, mid, i, val);
        else
            update(2 * id + 2, mid + 1, high, i, val);
        seg[id] = min(seg[2 * id + 1], seg[2 * id + 2]);
    }
};
```

2.9 Sparse table

```
vector<ll> lt(sz + 10);
void buildLogTable() {
    lt[1] = 0;
    for (ll i = 2; i < sz; i++)
        lt[i] = lt[i / 2] + 1;
}
struct sparseTable {
    ll n; vector<ll> v; vector<vector<ll>> gc;
    sparseTable(vector<ll> temp) {
        n = temp.size(); ll k = lt[n] + 1;
        gc.resize(n, vector<ll>(k)); v = temp; gcdBuild();
    }
    void gcdBuild() {
        for (ll i = 0; i < n; i++)
            gc[i][0] = v[i];
        for (ll k = 1; (1 << k) <= n; k++) {
            for (ll i = 0; i + (1 << k) <= n; i++) {
                gc[i][k] = GCD(gc[i][k - 1], gc[i + (1 << (k -
                    1))][k - 1]);
            }
        }
    }
    ll findGCD(ll st, ll en) {
        ll k = lt[en - st + 1];
        return GCD(gc[st][k], gc[en - (1 << k) + 1][k]);
    }
};
```

2.10 SquareRoot

```
void solve() { ll n; cin >> n; vector<ll> a(n); for (ll i =
    0; i < n; i++) cin >> a[i]; ll block_size = ceil(sqrt(n
     * 1.0)), blocks = ceil(n / (block_size * 1.0)); vector
    <ll> arr(blocks, -1); ll m; cin >> m; while (m--) { ll
    x; cin >> x; if (x == 2) { ll y; cin >> y; for (ll i =
    0; i < blocks; i++) arr[i] = max(arr[i], y); } else {
    ll p, y; cin >> p >> y; p--; ll bl = (p / block_size);
    ll l = (bl * block_size), r = min(((bl + 1) *
    block_size) - 1, n - 1); for (ll i = l; i <= r; i++) a[
    i] = max(a[i], arr[bl]); arr[bl] = -1; a[p] = y; } }
    for (ll i = 0; i < n; i++) { ll bl = (i / block_size);
    a[i] = max(a[i], arr[bl]); } for (ll i = 0; i < n; i++)
     { if (i > 0) cout << ' '; cout << a[i]; } cout << endl
    ; }
```

2.11 TernarySearch

```
double ternary_search(double l, double r) {
    double eps = 1e-9;          //set the error limit here
    while (r - l > eps) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;
        double f1 = f(m1);    //evaluates the function at m1
        double f2 = f(m2);    //evaluates the function at m2
        if (f1 < f2)
            l = m1;
        else
            r = m2;
    }
    return f(l);                //return the maximum of f(x)
        in [l, r]
}
```

2.12 next$_{s}$mallerpreviouss omaller

```
ll Next_smaller[N + 2], Prev_smaller[N + 2]; void
    NEXTSMALLER() { stack<int> st; for (int i = 1; i <= n;
    i++) { if (st.empty()) st.push(i); else { while (!st.
    empty() && a[st.top()] > a[i]) { Next_smaller[st.top()]
     = i; st.pop(); } st.push(i); } } while (!st.empty()) {
     Next_smaller[st.top()] = n + 1; st.pop(); } }
void PREVSMALLER() { stack<int> st; for (int i = n; i >= 1;
    i--) { if (st.empty()) st.push(i); else { while (!st.
    empty() && a[st.top()] > a[i]) { Prev_smaller[st.top()]
     = i; st.pop(); } st.push(i); } } while (!st.empty()) {
     Prev_smaller[st.top()] = 0; st.pop(); } }
```

2.13 ordered set

```
#include<ext/pb_ds/assoc_container.hpp>
#include<ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;
template<class T> using ordered_set =
    tree<T, null_type, less<T>, rb_tree_tag,
        tree_order_statistics_node_update>; // find_by_order
        , order_of_key
//1 2 2 3 3 7
//greater->>descending order => 7 3 2 1
//less ->>ascending order => 1 2 3 7
//less_equal -> ascending but in duplicate value => 1 2 2 3
    3 7 so this will work as multiset
// multi order set erase = --s.lower_bound(value)
ordered_set<int> A; // declaration
// Inserting elements - 1st query
cout << endl;
// finding kth element - 4th query
cout << "0th element: " << *A.find_by_order(0) << endl;

// finding number of elements smaller than X - 3rd query
cout << "No. of elems smaller than 6: " << A.order_of_key(6)
     << endl; // 2

// lower bound -> Lower Bound of X = first element >= X in
    the set
cout << "Lower Bound of 6: " << *A.lower_bound(6) << endl;
```

```
// Upper bound -> Upper Bound of X = first element > X in
    the set
cout << "Upper Bound of 6: " << *A.upper_bound(6) << endl;
```

2.14 segment tree persistent lazy

```
mint s[N], P = 37;
struct PST {
#define lc t[cur].l
#define rc t[cur].r
  struct node {
    int l = 0, r = 0, lazy = 0, p = 0;
    mint val = 0;
  } t[300 * N];
  int T = 0;
  int build(int b, int e) {
    int cur = ++T;
    if(b == e) return cur;
    int mid = b + e >> 1;
    lc = build(b, mid);
    rc = build(mid + 1, e);
    return cur;
  }
  int push(int pre, int b, int e, int x = 0) {
    int cur = ++T;
    t[cur] = t[pre];
    t[cur].lazy ^= x;
    if(t[cur].lazy) {
      t[cur].val = s[e] - s[b - 1] - t[cur].val;
      t[cur].p ^= 1;
      if(b != e) {
        lc = ++T;
        rc = ++T;
        t[lc] = t[t[pre].l];
        t[rc] = t[t[pre].r];
        t[lc].lazy ^= 1, t[rc].lazy ^= 1;
      }
    }
    t[cur].lazy = 0;
    return cur;
  }
  int upd(int pre, int b, int e, int i, int j) {
    int cur = push(pre, b, e);
    if(b > j || e < i) return cur;
    if(i <= b && e <= j) {
      cur = push(cur, b, e, 1);
      return cur;
    }
    int mid = b + e >> 1;
    lc = upd(lc, b, mid, i, j);
    rc = upd(rc, mid + 1, e, i, j);
    t[cur].val = t[lc].val + t[rc].val;
    return cur;
  }
  int cmp(int cur, int oth, int b, int e) {
    cur = push(cur, b, e);
    oth = push(oth, b, e);
    if(t[cur].val == t[oth].val) return 0;
    if(b == e) {
      int x = t[cur].p, y = t[oth].p;
      if(x == y) return 0;
      else if(x > y) return 1;
      else return 2;
    }
    int mid = b + e >> 1;
    int p = cmp(lc, t[oth].l, b, mid);
    if(p) return p;
    return cmp(rc, t[oth].r, mid + 1, e);
  }
  void print(int cur, int b, int e) {
    cur = push(cur, b, e);
    if(b == e) {
      cout << t[cur].p;
      return;
    }
    int mid = b + e >> 1;
    print(lc, b, mid);
    print(rc, mid + 1, e);
  }
} t;
int root[N];
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  for(int i = 1; i < N; i++) s[i] = s[i - 1] + P.pow(i);
  int n, q;
  cin >> n >> q;
  root[0] = t.build(1, n);
  int ans = 0;
  for(int i = 1; i <= q; i++) {
    int l, r;
    cin >> l >> r;
    root[i] = t.upd(root[i - 1], 1, n, l, r);
    if(t.cmp(root[ans], root[i], 1, n) == 2) ans = i;
  }
  t.print(root[ans], 1, n);
  return 0;
}
```

2.15 segment tree point update and max subarray sum

```
const ll N = 500000; // Define the maximum size of your
    array
ll a[N];
struct pr {
  ll prf = 0, suf = 0, sum = 0 , mx = 0;
};
struct SegmentTree {
#define lc (node << 1)
#define rc ((node << 1) | 1)
  vector<pr> tree;
  vector<ll> lazy;
  SegmentTree(){
    tree.resize(4 * N);
    lazy.resize(4 * N);
  }
  pr combine(pr a, pr b){
    pr c;
    c.prf = max(a.sum + b.prf, a.prf);
    c.suf = max(b.suf, b.sum + a.suf);
    c.sum = a.sum + b.sum;
    c.mx  = max({a.mx, b.mx, a.suf + b.prf});
    return c;
  }
  void build(ll node, ll b, ll e){
    if (b == e){
      tree[node].sum = a[b];
      tree[node].prf = max(0LL, a[b]);
      tree[node].suf = max(0LL, a[b]);
      tree[node].mx = max(0LL, a[b]);
      lazy[node] = 0;
      return;
    }
    ll mid = (b + e) >> 1;
    build(lc, b, mid);
    build(rc, mid + 1, e);
    lazy[node] = 0;
    tree[node] = combine(tree[lc], tree[rc]);
  }
  void update(ll node, ll b, ll e, ll l, ll x) {
    if (l > e || l < b)
      return;
    if (b == e && b == l){
      tree[node] = {max(x, 0LL), max(x, 0LL), x, max(x, 0LL)
          };
      return;
    }
    ll mid = (b + e) >> 1;
```

```
    update(lc, b, mid, l, x);
    update(rc, mid + 1, e, l, x);
    tree[node] = combine(tree[lc], tree[rc]);
  }
  pr query(ll node, ll b, ll e, ll l, ll r) {
    if (l > e || r < b)
      return {0, 0, 0, 0};
    if (l <= b && r >= e){
      return tree[node];
    }
    ll mid = (b + e) >> 1;
    return combine(query(lc, b, mid, l, r) , query(rc, mid +
        1, e, l, r));
  }
};
```

2.16 wavelet tree

```
const int MAXN = (int)3e5 + 9;
const int MAXV = (int)1e9 + 9; //maximum value of any
    element in array
//array values can be negative too, use appropriate minimum
    and maximum value
struct wavelet_tree {
    int lo, hi;
    wavelet_tree *l, *r;
    int *b, *c, bsz, csz; // c holds the prefix sum of
        elements
    wavelet_tree() {
        lo = 1; hi = 0; bsz = 0; csz = 0, l = NULL; r = NULL;
    }
    void init(int *from, int *to, int x, int y) {
        lo = x, hi = y;
        if (from >= to) return;
        int mid = (lo + hi) >> 1;
        auto f = [mid](int x) {
            return x <= mid;
        };
        b = (int*)malloc((to - from + 2) * sizeof(int)); bsz
            = 0; b[bsz++] = 0;
        c = (int*)malloc((to - from + 2) * sizeof(int)); csz
            = 0; c[csz++] = 0;
        for (auto it = from; it != to; it++) {
            b[bsz] = (b[bsz - 1] + f(*it));
            c[csz] = (c[csz - 1] + (*it));
            bsz++; csz++;
        }
        if (hi == lo) return;
        auto pivot = stable_partition(from, to, f);
        l = new wavelet_tree();
        l->init(from, pivot, lo, mid);
        r = new wavelet_tree();
        r->init(pivot, to, mid + 1, hi);
    }
    //kth smallest element in [l, r]
    //for array [1,2,1,3,5] 2nd smallest is 1 and 3rd
        smallest is 2
    int kth(int l, int r, int k) {
        if (l > r) return 0;
        if (lo == hi) return lo;
        int inLeft = b[r] - b[l - 1], lb = b[l - 1], rb = b[r
            ];
        if (k <= inLeft) return this->l->kth(lb + 1, rb, k);
        return this->r->kth(l - lb, r - rb, k - inLeft);
    }
    //count of numbers in [l, r] Less than or equal to k
    int LTE(int l, int r, int k) {
        if (l > r || k < lo) return 0;
        if (hi <= k) return r - l + 1;
        int lb = b[l - 1], rb = b[r];
        return this->l->LTE(lb + 1, rb, k) + this->r->LTE(l -
            lb, r - rb, k);
    }
    //count of numbers in [l, r] equal to k
    int count(int l, int r, int k) {
        if (l > r || k < lo || k > hi) return 0;
        if (lo == hi) return r - l + 1;
        int lb = b[l - 1], rb = b[r];
        int mid = (lo + hi) >> 1;
        if (k <= mid) return this->l->count(lb + 1, rb, k);
        return this->r->count(l - lb, r - rb, k);
    }
    //sum of numbers in [l ,r] less than or equal to k
    int sum(int l, int r, int k) {
        if (l > r or k < lo) return 0;
        if (hi <= k) return c[r] - c[l - 1];
        int lb = b[l - 1], rb = b[r];
        return this->l->sum(lb + 1, rb, k) + this->r->sum(l -
            lb, r - rb, k);
    }
    ~wavelet_tree() {
        delete l; delete r;
    }
};
wavelet_tree t;
int a[MAXN];
int main() {
    int i, j, k, n, m, q, l, r;
    cin >> n;
    for (i = 1; i <= n; i++) cin >> a[i];
    t.init(a + 1, a + n + 1, -MAXV, MAXV);
    //beware! after the init() operation array a[] will not
        be same
    cin >> q;
    while (q--) {
        int x;
        cin >> x;
        cin >> l >> r >> k;
        if (x == 0) {
            //kth smallest
            cout << t.kth(l, r, k) << endl;
        } else if (x == 1) {
            //less than or equal to K
            cout << t.LTE(l, r, k) << endl;
        } else if (x == 2) {
            //count occurence of K in [l, r]
            cout << t.count(l, r, k) << endl;
        }
        if (x == 3) {
            //sum of elements less than or equal to K in [l,
                r]
            cout << t.sum(l, r, k) << endl;
        }
    }
    return 0;
}
```

2.17 xor basis

```
struct XorBasis {
    vector<ll> basis; ll N = 0, tmp = 0;
    // Adds a new number to the basis
    void add(ll x) { N++, tmp |= x; for (auto &i : basis) x =
        min(x, x ^ i); if (!x) return; for (auto &i : basis
        ) if ((i ^ x) < i) i ^= x; basis.push_back(x); sort(
        basis.begin(), basis.end()); }
    // Returns the size of the basis
    ll size() { return (ll)basis.size(); }
    // Clears the basis
    void clear() { N = 0, tmp = 0; basis.clear(); }
    // Checks if a given value can be formed from the basis
    bool possible(ll x) { for (auto &i : basis) x = min(x, x
        ^ i); return !x; }
    // Returns the maximum XOR achievable with the basis
    ll maxxor(ll x = 0) { for (auto &i : basis) x = max(x, x
        ^ i); return x; }
    // Returns the minimum XOR achievable with the basis
```

```
    ll minxor(ll x = 0) { for (auto &i : basis) x = min(x, x
        ^ i); return x; }
    // Returns the number of XOR values that can be formed
        with the basis
    ll cntxor(ll x) { if (!possible(x)) return 0LL; ll ans =
        1LL; for (int i = 0; i < N - size(); i++) ans = (ans
         * 2) % MOD; return ans; }
    // Returns the sum of all possible XORs that can be
        formed
    ll sumOfAll() { return tmp * (1LL << (N - 1)); }
    // Returns the k-th smallest XOR combination
    ll kth(ll k) { ll sz = size(); if (k > (1LL << sz))
        return -1; k--; ll ans = 0; for (ll i = 0; i < sz; i
        ++) if (k >> i & 1) ans ^= basis[i]; return ans; }
} xb;
```

3 Geo

3.1 Geo

```
#include <bits/stdc++.h>using namespace std;
// https://victorlecomte.com/cp-geo.pdf
const int N = 3e5 + 9;const double inf = 1e100;const double
    eps = 1e-9;const double PI = acos((double)-1.0);
int sign(double x) { return (x > eps) - (x < -eps); }
struct PT{double x, y;PT() { x = 0, y = 0; }PT(double x,
    double y) : x(x), y(y) {}PT(const PT &p) : x(p.x), y(p.
    y) {}PT operator+(const PT &a) const { return PT(x + a.
    x, y + a.y); }
PT operator-(const PT &a) const { return PT(x - a.x, y - a.y
    ); }PT operator*(const double a) const { return PT(x *
    a, y * a); }
friend PT operator*(const double &a, const PT &b) { return
    PT(a * b.x, a * b.y); }PT operator/(const double a)
    const { return PT(x / a, y / a); }bool operator==(PT a)
     const { return sign(a.x - x) == 0 && sign(a.y - y) ==
    0; }
bool operator!=(PT a) const { return !(*this == a); }bool
    operator<(PT a) const { return sign(a.x - x) == 0 ? y <
     a.y : x < a.x; }
bool operator>(PT a) const { return sign(a.x - x) == 0 ? y >
     a.y : x > a.x; }double norm() { return sqrt(x * x + y
    * y); }
double norm2() { return x * x + y * y; }PT perp() { return
    PT(-y, x); }double arg() { return atan2(y, x); }
PT truncate(double r){ // returns a vector with norm r and
    having same directiondouble k = norm();if (!sign(k))
    return *this;r /= k;return PT(x * r, y * r);}};
istream &operator>>(istream &in, PT &p) { return in >> p.x
    >> p.y; }
ostream &operator<<(ostream &out, PT &p) { return out << "("
     << p.x << "," << p.y << ")"; }
inline double dot(PT a, PT b) { return a.x * b.x + a.y * b.y
    ; }
inline double dist2(PT a, PT b) { return dot(a - b, a - b);
    }
inline double dist(PT a, PT b) { return sqrt(dot(a - b, a -
    b)); }
inline double cross(PT a, PT b) { return a.x * b.y - a.y * b
    .x; }
inline double cross2(PT a, PT b, PT c) { return cross(b - a,
     c - a); }
inline int orientation(PT a, PT b, PT c) { return sign(cross
    (b - a, c - a)); }
PT perp(PT a) { return PT(-a.y, a.x); }
PT rotateccw90(PT a) { return PT(-a.y, a.x); }
PT rotatecw90(PT a) { return PT(a.y, -a.x); }
PT rotateccw(PT a, double t) { return PT(a.x * cos(t) - a.y
    * sin(t), a.x * sin(t) + a.y * cos(t)); }
PT rotatecw(PT a, double t) { return PT(a.x * cos(t) + a.y *
     sin(t), -a.x * sin(t) + a.y * cos(t)); }
double SQ(double x) { return x * x; }
double rad_to_deg(double r) { return (r * 180.0 / PI); }
double deg_to_rad(double d) { return (d * PI / 180.0); }
double get_angle(PT a, PT b){double costheta = dot(a, b) / a
    .norm() / b.norm();return acos(max((double)-1.0, min((
    double)1.0, costheta)));}
bool is_point_in_angle(PT b, PT a, PT c, PT p){ // does
    point p lie in angle <bac
assert(orientation(a, b, c) != 0);if (orientation(a, c, b) <
     0)swap(b, c);
return orientation(a, c, p) >= 0 && orientation(a, b, p) <=
    0;}
bool half(PT p){return p.y > 0.0 || (p.y == 0.0 && p.x <
    0.0);}
void polar_sort(vector<PT> &v){ // sort points in
    counterclockwise
sort(v.begin(), v.end(), [](PT a, PT b){ return make_tuple(
    half(a), 0.0, a.norm2()) < make_tuple(half(b), cross(a,
     b), b.norm2()); });}
void polar_sort(vector<PT> &v, PT o){ // sort points in
    counterclockwise with respect to point o
sort(v.begin(), v.end(), [&](PT a, PT b){ return make_tuple(
    half(a - o), 0.0, (a - o).norm2()) < make_tuple(half(b
    - o), cross(a - o, b - o), (b - o).norm2()); });}
struct line{PT a, b; // goes through points a and bPT v;
    double c; // line form: direction vec [cross] (x, y) =
    c
line() {}// direction vector v and offset cline(PT v, double
     c) : v(v), c(c){auto p = get_points();a = p.first;b =
    p.second;}
// equation ax + by + c = 0
line(double _a, double _b, double _c) : v({_b, -_a}), c(-_c)
    {auto p = get_points();a = p.first;b = p.second;}
// goes through points p and q
line(PT p, PT q) : v(q - p), c(cross(v, p)), a(p), b(q) {}
pair<PT, PT> get_points()
{ // extract any two points from this linePT p, q;double a =
     -v.y, b = v.x; // ax + by = cif (sign(a) == 0){p = PT
    (0, c / b);q = PT(1, c / b);}
else if (sign(b) == 0){p = PT(c / a, 0);q = PT(c / a, 1);}
else{p = PT(0, c / b);q = PT(1, (c - a) / b);}return {p, q
    };}
// ax + by + c = 0array<double, 3> get_abc(){double a = -v.y
    , b = v.x;return {a, b, -c};}
// 1 if on the left, -1 if on the right, 0 if on the line
int side(PT p) { return sign(cross(v, p) - c); }
// line that is perpendicular to this and goes through point
     p
line perpendicular_through(PT p) { return {p, p + perp(v)};
    }
// translate the line by vector t i.e. shifting it by vector
     t
line translate(PT t) { return {v, c + cross(v, t)}; }
// compare two points by their orthogonal projection on this
     line
// a projection point comes before another if it comes first
     according to vector v
bool cmp_by_projection(PT p, PT q) { return dot(v, p) < dot(
    v, q); }
line shift_left(double d){PT z = v.perp().truncate(d);return
     line(a + z, b + z);}};
// find a point from a through b with distance d
PT point_along_line(PT a, PT b, double d){assert(a != b);
    return a + (((b - a) / (b - a).norm()) * d);}
// projection point c onto line through a and b assuming a
    != b
PT project_from_point_to_line(PT a, PT b, PT c){return a + (
    b - a) * dot(c - a, b - a) / (b - a).norm2();}
// reflection point c onto line through a and b assuming a
    != b
PT reflection_from_point_to_line(PT a, PT b, PT c){PT p =
    project_from_point_to_line(a, b, c);return p + p - c;}
// minimum distance from point c to line through a and b
double dist_from_point_to_line(PT a, PT b, PT c){return fabs
    (cross(b - a, c - a) / (b - a).norm());}
// returns true if point p is on line segment ab
```

```
bool is_point_on_seg(PT a, PT b, PT p){if (fabs(cross(p - b,
     a - b)) < eps){
if (p.x < min(a.x, b.x) - eps || p.x > max(a.x, b.x) + eps)
     return false;
if (p.y < min(a.y, b.y) - eps || p.y > max(a.y, b.y) + eps)
     return false;return true;}return false;}
// minimum distance point from point c to segment ab that
     lies on segment ab
PT project_from_point_to_seg(PT a, PT b, PT c){double r =
     dist2(a, b);if (sign(r) == 0)return a;
r = dot(c - a, b - a) / r;if (r < 0)return a;if (r > 1)
     return b;return a + (b - a) * r;}
// minimum distance from point c to segment ab
double dist_from_point_to_seg(PT a, PT b, PT c){return dist(
     c, project_from_point_to_seg(a, b, c));}
// 0 if not parallel, 1 if parallel, 2 if collinear
int is_parallel(PT a, PT b, PT c, PT d){double k = fabs(
     cross(b - a, d - c));
if (k < eps){if (fabs(cross(a - b, a - c)) < eps && fabs(
     cross(c - d, c - a)) < eps)return 2;
else return 1;}else return 0;}
// check if two lines are same
bool are_lines_same(PT a, PT b, PT c, PT d){
if (fabs(cross(a - c, c - d)) < eps && fabs(cross(b - c, c -
      d)) < eps)return true;return false;}
// bisector vector of <abc
PT angle_bisector(PT &a, PT &b, PT &c){PT p = a - b, q = c -
      b;return p + q * sqrt(dot(p, p) / dot(q, q));}
// 1 if point is ccw to the line, 2 if point is cw to the
     line, 3 if point is on the line
int point_line_relation(PT a, PT b, PT p){int c = sign(cross
     (p - a, b - a));
if (c < 0)return 1;if (c > 0)return 2;return 3;}
// intersection point between ab and cd assuming unique
     intersection exists
bool line_line_intersection(PT a, PT b, PT c, PT d, PT &ans)
     {
double a1 = a.y - b.y, b1 = b.x - a.x, c1 = cross(a, b);
     double a2 = c.y - d.y, b2 = d.x - c.x, c2 = cross(c, d)
     ;
double det = a1 * b2 - a2 * b1;if (det == 0)return 0;
ans = PT((b1 * c2 - b2 * c1) / det, (c1 * a2 - a1 * c2) /
     det);return 1;}
// intersection point between segment ab and segment cd
     assuming unique intersection exists
bool seg_seg_intersection(PT a, PT b, PT c, PT d, PT &ans){
double oa = cross2(c, d, a), ob = cross2(c, d, b);double oc
     = cross2(a, b, c), od = cross2(a, b, d);if (oa * ob < 0
      && oc * od < 0)
{ans = (a * ob - b * oa) / (ob - oa);return 1;}elsereturn
     0;}
// intersection point between segment ab and segment cd
     assuming unique intersection may not exists
// se.size()==0 means no intersection
// se.size()==1 means one intersection
// se.size()==2 means range intersection
set<PT> seg_seg_intersection_inside(PT a, PT b, PT c, PT d){
PT ans;if (seg_seg_intersection(a, b, c, d, ans))return {ans
     };set<PT> se;
if (is_point_on_seg(c, d, a))se.insert(a);if (
     is_point_on_seg(c, d, b))se.insert(b);
if (is_point_on_seg(a, b, c))se.insert(c);
if (is_point_on_seg(a, b, d))se.insert(d);return se;}
// intersection between segment ab and line cd
// 0 if do not intersect, 1 if proper intersect, 2 if
     segment intersect
int seg_line_relation(PT a, PT b, PT c, PT d){
double p = cross2(c, d, a);double q = cross2(c, d, b);
if (sign(p) == 0 && sign(q) == 0)return 2;
else if (p * q < 0)return 1;else return 0;}
// intersection between segament ab and line cd assuming
     unique intersection exists
bool seg_line_intersection(PT a, PT b, PT c, PT d, PT &ans){
bool k = seg_line_relation(a, b, c, d);assert(k != 2);if (k)
     line_line_intersection(a, b, c, d, ans);return k;}
// minimum distance from segment ab to segment cd
double dist_from_seg_to_seg(PT a, PT b, PT c, PT d){PT dummy
     ;if (seg_seg_intersection(a, b, c, d, dummy))return
     0.0;
else return min({dist_from_point_to_seg(a, b, c),
     dist_from_point_to_seg(a, b, d),dist_from_point_to_seg(
     c, d, a), dist_from_point_to_seg(c, d, b)});}
// minimum distance from point c to ray (starting point a
     and direction vector b)
double dist_from_point_to_ray(PT a, PT b, PT c){b = a + b;
     double r = dot(c - a, b - a);if (r < 0.0)return dist(c,
      a);return dist_from_point_to_line(a, b, c);}
// starting point as and direction vector ad
bool ray_ray_intersection(PT as, PT ad, PT bs, PT bd){
double dx = bs.x - as.x, dy = bs.y - as.y;double det = bd.x
     * ad.y - bd.y * ad.x;if (fabs(det) < eps)return 0;
     double u = (dy * bd.x - dx * bd.y) / det;
double v = (dy * ad.x - dx * ad.y) / det;
if (sign(u) >= 0 && sign(v) >= 0)return 1;elsereturn 0;}
double ray_ray_distance(PT as, PT ad, PT bs, PT bd){if (
     ray_ray_intersection(as, ad, bs, bd))return 0.0;
double ans = dist_from_point_to_ray(as, ad, bs);ans = min(
     ans, dist_from_point_to_ray(bs, bd, as));return ans;}
struct circle{PT p;double r;
circle() {}circle(PT _p, double _r) : p(_p), r(_r) {};
// center (x, y) and radius r
circle(double x, double y, double _r) : p(PT(x, y)), r(_r)
     {};
// circumcircle of a triangle
// the three points must be unique
circle(PT a, PT b, PT c){b = (a + b) * 0.5;c = (a + c) *
     0.5;line_line_intersection(b, b + rotatecw90(a - b), c,
      c + rotatecw90(a - c), p);r = dist(a, p);}
// inscribed circle of a triangle
// pass a bool just to differentiate from circumcircle
circle(PT a, PT b, PT c, bool t){line u, v;double m = atan2(
     b.y - a.y, b.x - a.x), n = atan2(c.y - a.y, c.x - a.x);
u.a = a;u.b = u.a + (PT(cos((n + m) / 2.0), sin((n + m) /
     2.0)));v.a = b;
m = atan2(a.y - b.y, a.x - b.x), n = atan2(c.y - b.y, c.x -
     b.x);v.b = v.a + (PT(cos((n + m) / 2.0), sin((n + m) /
     2.0)));
line_line_intersection(u.a, u.b, v.a, v.b, p);r =
     dist_from_point_to_seg(a, b, p);}
bool operator==(circle v) { return p == v.p && sign(r - v.r)
      == 0; }double area() { return PI * r * r; }double
     circumference() { return 2.0 * PI * r; }};
// 0 if outside, 1 if on circumference, 2 if inside circle
int circle_point_relation(PT p, double r, PT b){double d =
     dist(p, b);if (sign(d - r) < 0)return 2;if (sign(d - r)
      == 0)return 1;return 0;}
// 0 if outside, 1 if on circumference, 2 if inside circle
int circle_line_relation(PT p, double r, PT a, PT b){double
     d = dist_from_point_to_line(a, b, p);if (sign(d - r) <
     0)return 2;
if (sign(d - r) == 0)return 1;return 0;}
// compute intersection of line through points a and b with
// circle centered at c with radius r > 0
vector<PT> circle_line_intersection(PT c, double r, PT a, PT
      b){vector<PT> ret;
b = b - a;a = a - c;double A = dot(b, b), B = dot(a, b);
     double C = dot(a, a) - r * r, D = B * B - A * C;
if (D < -eps)return ret;
ret.push_back(c + a + b * (-B + sqrt(D + eps)) / A);
if (D > eps)ret.push_back(c + a + b * (-B - sqrt(D)) / A);
     return ret;}
// 5 - outside and do not intersect
// 4 - intersect outside in one point
// 3 - intersect in 2 points
// 2 - intersect inside in one point
// 1 - inside and do not intersect
int circle_circle_relation(PT a, double r, PT b, double R){
     double d = dist(a, b);
if (sign(d - r - R) > 0)return 5;
```

```
if (sign(d - r - R) == 0)return 4;
double l = fabs(r - R);
if (sign(d - r - R) < 0 && sign(d - l) > 0)return 3;
if (sign(d - l) == 0)return 2;
if (sign(d - l) < 0)return 1;assert(0);return -1;}
vector<PT> circle_circle_intersection(PT a, double r, PT b,
    double R){
if (a == b && sign(r - R) == 0)return {PT(1e18, 1e18)};
    vector<PT> ret;double d = sqrt(dist2(a, b));
if (d > r + R || d + min(r, R) < max(r, R))return ret;double
     x = (d * d - R * R + r * r) / (2 * d);
double y = sqrt(r * r - x * x);PT v = (b - a) / d;
ret.push_back(a + v * x + rotateccw90(v) * y);if (y > 0)ret.
    push_back(a + v * x - rotateccw90(v) * y);return ret;}
// returns two circle c1, c2 through points a, b and of
    radius r
// 0 if there is no such circle, 1 if one circle, 2 if two
    circle
int get_circle(PT a, PT b, double r, circle &c1, circle &c2)
    {vector<PT> v = circle_circle_intersection(a, r, b, r);
int t = v.size();if (!t)return 0;
c1.p = v[0], c1.r = r;if (t == 2)c2.p = v[1], c2.r = r;
    return t;}
// returns two circle c1, c2 which is tangent to line u,
    goes through
// point q and has radius r1; 0 for no circle, 1 if c1 = c2
    , 2 if c1 != c2
int get_circle(line u, PT q, double r1, circle &c1, circle &
    c2){
double d = dist_from_point_to_line(u.a, u.b, q);if (sign(d -
     r1 * 2.0) > 0)return 0;
if (sign(d) == 0){cout << u.v.x << ' ' << u.v.y << '\n';
c1.p = q + rotateccw90(u.v).truncate(r1);c2.p = q +
    rotatecw90(u.v).truncate(r1);c1.r = c2.r = r1;return
    2;}
line u1 = line(u.a + rotateccw90(u.v).truncate(r1), u.b +
    rotateccw90(u.v).truncate(r1));line u2 = line(u.a +
    rotatecw90(u.v).truncate(r1), u.b + rotatecw90(u.v).
    truncate(r1));
circle cc = circle(q, r1);PT p1, p2;vector<PT> v;v =
    circle_line_intersection(q, r1, u1.a, u1.b);
if (!v.size())v = circle_line_intersection(q, r1, u2.a, u2.b
    );v.push_back(v[0]);p1 = v[0], p2 = v[1];
c1 = circle(p1, r1);if (p1 == p2){c2 = c1;return 1;}c2 =
    circle(p2, r1);return 2;}
// returns the circle such that for all points w on the
    circumference of the circle
// dist(w, a) : dist(w, b) = rp : rq
// rp != rq
// https://en.wikipedia.org/wiki/Circles_of_Apollonius
circle get_apollonius_circle(PT p, PT q, double rp, double
    rq){rq *= rq;rp *= rp;double a = rq - rp;
assert(sign(a));double g = rq * p.x - rp * q.x;g /= a;double
     h = rq * p.y - rp * q.y;h /= a;double c = rq * p.x * p
    .x - rp * q.x * q.x + rq * p.y * p.y - rp * q.y * q.y;
c /= a;PT o(g, h);double r = g * g + h * h - c;r = sqrt(r);
    return circle(o, r);}
// returns area of intersection between two circles
double circle_circle_area(PT a, double r1, PT b, double r2){
    double d = (a - b).norm();if (r1 + r2 < d + eps)return
    0;
if (r1 + d < r2 + eps)return PI * r1 * r1;
if (r2 + d < r1 + eps)return PI * r2 * r2;
double theta_1 = acos((r1 * r1 + d * d - r2 * r2) / (2 * r1
    * d)),theta_2 = acos((r2 * r2 + d * d - r1 * r1) / (2 *
     r2 * d));
return r1 * r1 * (theta_1 - sin(2 * theta_1) / 2.) + r2 * r2
     * (theta_2 - sin(2 * theta_2) / 2.);}
// tangent lines from point q to the circle
int tangent_lines_from_point(PT p, double r, PT q, line &u,
    line &v){
int x = sign(dist2(p, q) - r * r);if (x < 0)return 0; //
    point in cricleif (x == 0)
{ // point on circle
u = line(q, q + rotateccw90(q - p));v = u;return 1;}
double d = dist(p, q);double l = r * r / d;double h = sqrt(r
     * r - l * l);u = line(q, p + ((q - p).truncate(l) + (
    rotateccw90(q - p).truncate(h))));
v = line(q, p + ((q - p).truncate(l) + (rotatecw90(q - p).
    truncate(h))));return 2;}
// returns outer tangents line of two circles
// if inner == 1 it returns inner tangent lines
int tangents_lines_from_circle(PT c1, double r1, PT c2,
    double r2, bool inner, line &u, line &v){
if (inner)r2 = -r2;PT d = c2 - c1;double dr = r1 - r2, d2 =
    d.norm2(), h2 = d2 - dr * dr;
if (d2 == 0 || h2 < 0){assert(h2 != 0);return 0;}vector<pair
    <PT, PT>> out;for (int tmp : {-1, 1}){PT v = (d * dr +
    rotateccw90(d) * sqrt(h2) * tmp) / d2;out.push_back({c1
     + v * r1, c2 + v * r2});}
u = line(out[0].first, out[0].second);
if (out.size() == 2)v = line(out[1].first, out[1].second);
    return 1 + (h2 > 0);}
// O(n^2 log n)
// https://vjudge.net/problem/UVA-12056
struct CircleUnion
{int n;double x[2020], y[2020], r[2020];int covered[2020];
    vector<pair<double, double>> seg, cover;double arc, pol
    ;inline int sign(double x) { return x < -eps ? -1 : x >
     eps; }inline int sign(double x, double y) { return
    sign(x - y); }
inline double SQ(const double x) { return x * x; }inline
    double dist(double x1, double y1, double x2, double y2)
     { return sqrt(SQ(x1 - x2) + SQ(y1 - y2)); }
inline double angle(double A, double B, double C)
{double val = (SQ(A) + SQ(B) - SQ(C)) / (2 * A * B);if (val
    < -1)val = -1;
if (val > +1)val = +1;return acos(val);}
CircleUnion(){n = 0;seg.clear(), cover.clear();arc = pol =
    0;}
void init(){n = 0;seg.clear(), cover.clear();arc = pol = 0;}
void add(double xx, double yy, double rr)
{x[n] = xx, y[n] = yy, r[n] = rr, covered[n] = 0, n++;}
void getarea(int i, double lef, double rig){
arc += 0.5 * r[i] * r[i] * (rig - lef - sin(rig - lef));
double x1 = x[i] + r[i] * cos(lef), y1 = y[i] + r[i] * sin(
    lef);
double x2 = x[i] + r[i] * cos(rig), y2 = y[i] + r[i] * sin(
    rig);pol += x1 * y2 - x2 * y1;}
double solve(){for (int i = 0; i < n; i++)
{for (int j = 0; j < i; j++){if (!sign(x[i] - x[j]) && !sign
    (y[i] - y[j]) && !sign(r[i] - r[j]))
{r[i] = 0.0;break;}}}
for (int i = 0; i < n; i++){for (int j = 0; j < n; j++){if (
    i != j && sign(r[j] - r[i]) >= 0 && sign(dist(x[i], y[i
    ], x[j], y[j]) - (r[j] - r[i])) <= 0){
covered[i] = 1;break;}}}for (int i = 0; i < n; i++){
if (sign(r[i]) && !covered[i]){seg.clear();
for (int j = 0; j < n; j++){if (i != j){
double d = dist(x[i], y[i], x[j], y[j]);
if (sign(d - (r[j] + r[i])) >= 0 || sign(d - abs(r[j] - r[i
    ])) <= 0)
{continue;}double alpha = atan2(y[j] - y[i], x[j] - x[i]);
double beta = angle(r[i], d, r[j]);pair<double, double> tmp(
    alpha - beta, alpha + beta);
if (sign(tmp.first) <= 0 && sign(tmp.second) <= 0){
seg.push_back(pair<double, double>(2 * PI + tmp.first, 2 *
    PI + tmp.second));}
else if (sign(tmp.first) < 0){seg.push_back(pair<double,
    double>(2 * PI + tmp.first, 2 * PI));seg.push_back(pair
    <double, double>(0, tmp.second));}
else{seg.push_back(tmp);}}}
sort(seg.begin(), seg.end());double rig = 0;
for (vector<pair<double, double>>::iterator iter = seg.begin
    (); iter != seg.end(); iter++){
if (sign(rig - iter->first) >= 0){rig = max(rig, iter->
    second);}
else{getarea(i, rig, iter->first);rig = iter->second;}}
if (!sign(rig)){arc += r[i] * r[i] * PI;}
```

```
else{getarea(i, rig, 2 * PI);}}}return pol / 2.0 + arc;}} CU
    ;
double area_of_triangle(PT a, PT b, PT c){return fabs(cross(
    b - a, c - a) * 0.5);}
// -1 if strictly inside, 0 if on the polygon, 1 if strictly
     outside
int is_point_in_triangle(PT a, PT b, PT c, PT p){
if (sign(cross(b - a, c - a)) < 0)swap(b, c);
int c1 = sign(cross(b - a, p - a));int c2 = sign(cross(c - b
    , p - b));int c3 = sign(cross(a - c, p - c));
if (c1 < 0 || c2 < 0 || c3 < 0)return 1;if (c1 + c2 + c3 !=
    3)return 0;return -1;}
double perimeter(vector<PT> &p){
double ans = 0;int n = p.size();
for (int i = 0; i < n; i++)ans += dist(p[i], p[(i + 1) % n])
    ;return ans;}
double area(vector<PT> &p){double ans = 0;int n = p.size();
    or (int i = 0; i < n; i++)ans += cross(p[i], p[(i + 1)
    % n]);return fabs(ans) * 0.5;}
// centroid of a (possibly non-convex) polygon,
// assuming that the coordinates are listed in a clockwise
    or
// counterclockwise fashion. Note that the centroid is often
     known as
// the "center of gravity" or "center of mass".
PT centroid(vector<PT> &p){int n = p.size();PT c(0, 0);
    double sum = 0;
for (int i = 0; i < n; i++)sum += cross(p[i], p[(i + 1) % n
    ]);double scale = 3.0 * sum;
for (int i = 0; i < n; i++){int j = (i + 1) % n;c = c + (p[i
    ] + p[j]) * cross(p[i], p[j]);}return c / scale;}
// 0 if cw, 1 if ccw
bool get_direction(vector<PT> &p){double ans = 0;int n = p.
    size();
for (int i = 0; i < n; i++)ans += cross(p[i], p[(i + 1) % n
    ]);if (sign(ans) > 0)return 1;return 0;}
// it returns a point such that the sum of distances
// from that point to all points in p is minimum
// O(n log^2 MX)
PT geometric_median(vector<PT> p){auto tot_dist = [&](PT z){
    double res = 0;
for (int i = 0; i < p.size(); i++)res += dist(p[i], z);
    return res;};
auto findY = [&](double x){double yl = -1e5, yr = 1e5;for (
    int i = 0; i < 60; i++){
double ym1 = yl + (yr - yl) / 3;double ym2 = yr - (yr - yl)
    / 3;double d1 = tot_dist(PT(x, ym1));
double d2 = tot_dist(PT(x, ym2));if (d1 < d2)yr = ym2;
else yl = ym1;}return pair<double, double>(yl, tot_dist(PT(x
    , yl)));};double xl = -1e5, xr = 1e5;
for (int i = 0; i < 60; i++){double xm1 = xl + (xr - xl) /
    3;
double xm2 = xr - (xr - xl) / 3;double y1, d1, y2, d2;auto z
     = findY(xm1);y1 = z.first;d1 = z.second;z = findY(xm2)
    ;
y2 = z.first;d2 = z.second;
if (d1 < d2)xr = xm2;elsexl = xm1;}return {xl, findY(xl).
    first};}
vector<PT> convex_hull(vector<PT> &p){
if (p.size() <= 1)return p;
vector<PT> v = p;sort(v.begin(), v.end());vector<PT> up, dn;
for (auto &p : v){
while (up.size() > 1 && orientation(up[up.size() - 2], up.
    back(), p) >= 0){up.pop_back();}
while (dn.size() > 1 && orientation(dn[dn.size() - 2], dn.
    back(), p) <= 0){dn.pop_back();}
up.push_back(p);dn.push_back(p);}
v = dn;if (v.size() > 1)v.pop_back();reverse(up.begin(), up.
    end());up.pop_back();
for (auto &p : up){v.push_back(p);}
if (v.size() == 2 && v[0] == v[1])v.pop_back();return v;}
// checks if convex or not
bool is_convex(vector<PT> &p){bool s[3];s[0] = s[1] = s[2] =
     0;int n = p.size();for (int i = 0; i < n; i++){int j =
     (i + 1) % n;int k = (j + 1) % n;
s[sign(cross(p[j] - p[i], p[k] - p[i])) + 1] = 1;if (s[0] &&
     s[2])return 0;}return 1;}
// -1 if strictly inside, 0 if on the polygon, 1 if strictly
     outside
// it must be strictly convex, otherwise make it strictly
    convex first
int is_point_in_convex(vector<PT> &p, const PT &x){ // O(log
     n)int n = p.size();assert(n >= 3);
int a = orientation(p[0], p[1], x), b = orientation(p[0], p[
    n - 1], x);if (a < 0 || b > 0)return 1;int l = 1, r = n
     - 1;while (l + 1 < r){
int mid = l + r >> 1;if (orientation(p[0], p[mid], x) >= 0)l
     = mid;elser = mid;}int k = orientation(p[l], p[r], x);
    if (k <= 0)return -k;
if (l == 1 && a == 0)return 0;if (r == n - 1 && b == 0)
    return 0;return -1;}
bool is_point_on_polygon(vector<PT> &p, const PT &z){int n =
     p.size();
for (int i = 0; i < n; i++){if (is_point_on_seg(p[i], p[(i +
     1) % n], z))return 1;}return 0;}
// returns 1e9 if the point is on the polygon
int winding_number(vector<PT> &p, const PT &z){ // O(n)
if (is_point_on_polygon(p, z))return 1e9;
int n = p.size(), ans = 0;
for (int i = 0; i < n; ++i){int j = (i + 1) % n;bool below =
     p[i].y < z.y;
if (below != (p[j].y < z.y)){
auto orient = orientation(z, p[j], p[i]);
if (orient == 0)return 0;
if (below == (orient > 0))ans += below ? 1 : -1;}}return ans
    ;}
// -1 if strictly inside, 0 if on the polygon, 1 if strictly
     outside
int is_point_in_polygon(vector<PT> &p, const PT &z){ // O(n)
int k = winding_number(p, z);return k == 1e9 ? 0 : k == 0 ?
    1: -1;}
// id of the vertex having maximum dot product with z
// polygon must need to be convex
// top - upper right vertex
// for minimum dot product negate z and return -dot(z, p[id
    ])
int extreme_vertex(vector<PT> &p, const PT &z, const int top
    ){ // O(log n)
int n = p.size();if (n == 1)return 0;
double ans = dot(p[0], z);int id = 0;
if (dot(p[top], z) > ans)ans = dot(p[top], z), id = top;
int l = 1, r = top - 1;
while (l < r){int mid = l + r >> 1;
if (dot(p[mid + 1], z) >= dot(p[mid], z))l = mid + 1;
else r = mid;}
if (dot(p[l], z) > ans)ans = dot(p[l], z), id = l;l = top +
    1, r = n - 1;while (l < r){
int mid = l + r >> 1;if (dot(p[(mid + 1) % n], z) >= dot(p[
    mid], z))l = mid + 1;elser = mid;}l %= n;
if (dot(p[l], z) > ans)ans = dot(p[l], z), id = l;return id
    ;}
// maximum distance from any point on the perimeter to
    another point on the perimeter
double diameter(vector<PT> &p){int n = (int)p.size();if (n
    == 1)return 0;if (n == 2)return dist(p[0], p[1]);double
     ans = 0;int i = 0, j = 1;
while (i < n){
while (cross(p[(i + 1) % n] - p[i], p[(j + 1) % n] - p[j])
    >= 0){ans = max(ans, dist2(p[i], p[j]));j = (j + 1) % n
    ;}
ans = max(ans, dist2(p[i], p[j]));i++;}return sqrt(ans);}
// minimum distance between two parallel lines (non
    necessarily axis parallel)
// such that the polygon can be put between the lines
double width(vector<PT> &p){int n = (int)p.size();if (n <=
    2)return 0;double ans = inf;int i = 0, j = 1;while (i <
     n){
while (cross(p[(i + 1) % n] - p[i], p[(j + 1) % n] - p[j])
    >= 0)j = (j + 1) % n;ans = min(ans,
```

```
    dist_from_point_to_line(p[i], p[(i + 1) % n], p[j]));i
    ++;}return ans;}
// minimum perimeter
double minimum_enclosing_rectangle(vector<PT> &p){int n = p.
    size();
if (n <= 2)return perimeter(p);int mndot = 0;double tmp =
    dot(p[1] - p[0], p[0]);
for (int i = 1; i < n; i++){if (dot(p[1] - p[0], p[i]) <=
    tmp){tmp = dot(p[1] - p[0], p[i]);mndot = i;}}
double ans = inf;int i = 0, j = 1, mxdot = 1;
while (i < n){PT cur = p[(i + 1) % n] - p[i];while (cross(
    cur, p[(j + 1) % n] - p[j]) >= 0)j = (j + 1) % n;while
    (dot(p[(mxdot + 1) % n], cur) >= dot(p[mxdot], cur))
    mxdot = (mxdot + 1) % n;
while (dot(p[(mndot + 1) % n], cur) <= dot(p[mndot], cur))
    mndot = (mndot + 1) % n;ans = min(ans, 2.0 * ((dot(p[
    mxdot], cur) / cur.norm() - dot(p[mndot], cur) / cur.
    norm()) + dist_from_point_to_line(p[i], p[(i + 1) % n],
     p[j])));i++;}return ans;}
// given n points, find the minimum enclosing circle of the
    points
// call convex_hull() before this for faster solution
// expected O(n)
circle minimum_enclosing_circle(vector<PT> &p){
    random_shuffle(p.begin(), p.end());int n = p.size();
    circle c(p[0], 0);
for (int i = 1; i < n; i++){if (sign(dist(c.p, p[i]) - c.r)
    > 0){c = circle(p[i], 0);
for (int j = 0; j < i; j++){
if (sign(dist(c.p, p[j]) - c.r) > 0){
c = circle((p[i] + p[j]) / 2, dist(p[i], p[j]) / 2);
for (int k = 0; k < j; k++){if (sign(dist(c.p, p[k]) - c.r)
    > 0){c = circle(p[i], p[j], p[k]);}}}}}}return c;}
// returns a vector with the vertices of a polygon with
    everything
// to the left of the line going from a to b cut away.
vector<PT> cut(vector<PT> &p, PT a, PT b){vector<PT> ans;int
     n = (int)p.size();
for (int i = 0; i < n; i++){double c1 = cross(b - a, p[i] -
    a);double c2 = cross(b - a, p[(i + 1) % n] - a);
if (sign(c1) >= 0)ans.push_back(p[i]);if (sign(c1 * c2) < 0)
    {if (!is_parallel(p[i], p[(i + 1) % n], a, b)){PT tmp;
    line_line_intersection(p[i], p[(i + 1) % n], a, b, tmp)
    ;ans.push_back(tmp);}}}return ans;}
// not necessarily convex, boundary is included in the
    intersection
// returns total intersected length
// it returns the sum of the lengths of the portions of the
    line that are inside the polygon
double polygon_line_intersection(vector<PT> p, PT a, PT b){
int n = p.size();p.push_back(p[0]);line l = line(a, b);
    double ans = 0.0;vector<pair<double, int>> vec;for (int
     i = 0; i < n; i++){int s1 = orientation(a, b, p[i]);
    int s2 = orientation(a, b, p[i + 1]);
if (s1 == s2)continue;line t = line(p[i], p[i + 1]);PT inter
     = (t.v * l.c - l.v * t.c) / cross(l.v, t.v);double tmp
     = dot(inter, l.v);int f;
if (s1 > s2)f = s1 && s2 ? 2 : 1;
else f = s1 && s2 ? -2 : -1;vec.push_back(make_pair((f > 0 ?
     tmp - eps : tmp + eps), f)); // keep eps very small
    like 1e-12}
sort(vec.begin(), vec.end());for (int i = 0, j = 0; i + 1 <
    (int)vec.size(); i++){j += vec[i].second;
if (j)ans += vec[i + 1].first - vec[i].first; // if this
    portion is inside the polygon// else ans = 0; // if we
    want the maximum intersected length which is totally
    inside the polygon, uncomment this and take the maximum
     of ans}
ans = ans / sqrt(dot(l.v, l.v));p.pop_back();return ans;}
// given a convex polygon p, and a line ab and the top
    vertex of the polygon
// returns the intersection of the line with the polygon
// it returns the indices of the edges of the polygon that
    are intersected by the line
// so if it returns i, then the line intersects the edge (p[
    i], p[(i + 1) % n])
array<int, 2> convex_line_intersection(vector<PT> &p, PT a,
    PT b, int top){int end_a = extreme_vertex(p, (a - b).
    perp(), top);int end_b = extreme_vertex(p, (b - a).perp
    (), top);
auto cmp_l = [&](int i){ return orientation(a, p[i], b); };
    if (cmp_l(end_a) < 0 || cmp_l(end_b) > 0)return {-1,
    -1}; // no intersectionarray<int, 2> res;for (int i =
    0; i < 2; i++){int lo = end_b, hi = end_a, n = p.size()
    ;
while ((lo + 1) % n != hi){int m = ((lo + hi + (lo < hi ? 0
    : n)) / 2) % n;(cmp_l(m) == cmp_l(end_b) ? lo : hi) = m
    ;}res[i] = (lo + !cmp_l(hi)) % n;
swap(end_a, end_b);}
if (res[0] == res[1])return {res[0], -1}; // touches the
    vertex res[0]if (!cmp_l(res[0]) && !cmp_l(res[1]))
switch ((res[0] - res[1] + (int)p.size() + 1) % p.size()){
    case 0:return {res[0], res[0]}; // touches the edge (
    res[0], res[0] + 1)case 2:return {res[1], res[1]}; //
    touches the edge (res[1], res[1] + 1)}
return res; // intersects the edges (res[0], res[0] + 1) and
     (res[1], res[1] + 1)}

pair<PT, int> point_poly_tangent(vector<PT> &p, PT Q, int
    dir, int l, int r){
while (r - l > 1){int mid = (l + r) >> 1;bool pvs =
    orientation(Q, p[mid], p[mid - 1]) != -dir;
bool nxt = orientation(Q, p[mid], p[mid + 1]) != -dir;
if (pvs && nxt)return {p[mid], mid};
if (!(pvs || nxt)){auto p1 = point_poly_tangent(p, Q, dir,
    mid + 1, r);
auto p2 = point_poly_tangent(p, Q, dir, l, mid - 1);
return orientation(Q, p1.first, p2.first) == dir ? p1 : p2;}
if (!pvs){if (orientation(Q, p[mid], p[l]) == dir)r = mid -
    1;
else if (orientation(Q, p[l], p[r]) == dir)r = mid - 1;elsel
     = mid + 1;}
if (!nxt){if (orientation(Q, p[mid], p[l]) == dir)l = mid +
    1;
else if (orientation(Q, p[l], p[r]) == dir)r = mid - 1;
else l = mid + 1;}}
pair<PT, int> ret = {p[l], l};
for (int i = l + 1; i <= r; i++)ret = orientation(Q, ret.
    first, p[i]) != dir ? make_pair(p[i], i) : ret;
return ret;}
// (ccw, cw) tangents from a point that is outside this
    convex polygon
// returns indexes of the points
// ccw means the tangent from Q to that point is in the same
     direction as the polygon ccw direction
pair<int, int> tangents_from_point_to_polygon(vector<PT> &p,
     PT Q){int ccw = point_poly_tangent(p, Q, 1, 0, (int)p.
    size() - 1).second;int cw = point_poly_tangent(p, Q,
    -1, 0, (int)p.size() - 1).second;return make_pair(ccw,
    cw);}
// minimum distance from a point to a convex polygon
// it assumes point lie strictly outside the polygon
double dist_from_point_to_polygon(vector<PT> &p, PT z){
    double ans = inf;int n = p.size();if (n <= 3){for (int
    i = 0; i < n; i++)ans = min(ans, dist_from_point_to_seg
    (p[i], p[(i + 1) % n], z));return ans;}
auto [r, l] = tangents_from_point_to_polygon(p, z);if (l > r
    )r += n;
while (l < r){int mid = (l + r) >> 1;double left = dist2(p[
    mid % n], z), right = dist2(p[(mid + 1) % n], z);ans =
    min({ans, left, right});if (left < right)r = mid;elsel
    = mid + 1;}
ans = sqrt(ans);ans = min(ans, dist_from_point_to_seg(p[l %
    n], p[(l + 1) % n], z));ans = min(ans,
    dist_from_point_to_seg(p[l % n], p[(l - 1 + n) % n], z)
    );return ans;}
// minimum distance from convex polygon p to line ab
// returns 0 is it intersects with the polygon
// top - upper right vertex
```

```
double dist_from_polygon_to_line(vector<PT> &p, PT a, PT b,
     int top){ // O(log n)
PT orth = (b - a).perp();
if (orientation(a, b, p[0]) > 0)orth = (a - b).perp();
int id = extreme_vertex(p, orth, top);if (dot(p[id] - a,
     orth) > 0)return 0.0;// if orth and a are in the same
     half of the line, then poly and line intersects
return dist_from_point_to_line(a, b, p[id]); // does not
     intersect}
// minimum distance from a convex polygon to another convex
     polygon
// the polygon doesnot overlap or touch
// tested in https://toph.co/p/the-wall
double dist_from_polygon_to_polygon(vector<PT> &p1, vector<
     PT> &p2){ // O(n log n)
double ans = inf;
for (int i = 0; i < p1.size(); i++){ans = min(ans,
     dist_from_point_to_polygon(p2, p1[i]));}
for (int i = 0; i < p2.size(); i++){ans = min(ans,
     dist_from_point_to_polygon(p1, p2[i]));}
return ans;}
// maximum distance from a convex polygon to another convex
     polygon
double maximum_dist_from_polygon_to_polygon(vector<PT> &u,
     vector<PT> &v){ // O(n)
int n = (int)u.size(), m = (int)v.size();double ans = 0;
if (n < 3 || m < 3){
for (int i = 0; i < n; i++){
for (int j = 0; j < m; j++)ans = max(ans, dist2(u[i], v[j]))
     ;}return sqrt(ans);}
if (u[0].x > v[0].x)swap(n, m), swap(u, v);int i = 0, j = 0,
      step = n + m + 10;
while (j + 1 < m && v[j].x < v[j + 1].x)j++;
while (step--){if (cross(u[(i + 1) % n] - u[i], v[(j + 1) %
     m] - v[j]) >= 0)j = (j + 1) % m;elsei = (i + 1) % n;ans
      = max(ans, dist2(u[i], v[j]));}
return sqrt(ans);}

// calculates the area of the union of n polygons (not
     necessarily convex).
// the points within each polygon must be given in CCW order
     .
// complexity: O(N^2), where N is the total number of points
double rat(PT a, PT b, PT p){return !sign(a.x - b.x) ? (p.y
     - a.y) / (b.y - a.y) : (p.x - a.x) / (b.x - a.x);};
double polygon_union(vector<vector<PT>> &p){
int n = p.size();double ans = 0;
for (int i = 0; i < n; ++i){
for (int v = 0; v < (int)p[i].size(); ++v){
PT a = p[i][v], b = p[i][(v + 1) % p[i].size()];vector<pair<
     double, int>> segs;segs.emplace_back(0, 0), segs.
     emplace_back(1, 0);
for (int j = 0; j < n; ++j){
if (i != j){for (size_t u = 0; u < p[j].size(); ++u){
PT c = p[j][u], d = p[j][(u + 1) % p[j].size()];int sc =
     sign(cross(b - a, c - a)), sd = sign(cross(b - a, d - a
     ));if (!sc && !sd){
if (sign(dot(b - a, d - c)) > 0 && i > j){segs.emplace_back(
     rat(a, b, c), 1), segs.emplace_back(rat(a, b, d), -1)
     ;}}else{
double sa = cross(d - c, a - c), sb = cross(d - c, b - c);
if (sc >= 0 && sd < 0)segs.emplace_back(sa / (sa - sb), 1);
     else if (sc < 0 && sd >= 0)segs.emplace_back(sa / (sa -
      sb), -1);}}}}
sort(segs.begin(), segs.end());double pre = min(max(segs[0].
     first, 0.0), 1.0), now, sum = 0;int cnt = segs[0].
     second;
for (int j = 1; j < segs.size(); ++j){now = min(max(segs[j].
     first, 0.0), 1.0);if (!cnt)sum += now - pre;cnt += segs
     [j].second;pre = now;}ans += cross(a, b) * sum;}}return
      ans * 0.5;}
// contains all points p such that: cross(b - a, p - a) >= 0
struct HP{PT a, b;HP() {}HP(PT a, PT b) : a(a), b(b) {}HP(
     const HP &rhs) : a(rhs.a), b(rhs.b) {}
int operator<(const HP &rhs) const{PT p = b - a;PT q = rhs.b
      - rhs.a;
int fp = (p.y < 0 || (p.y == 0 && p.x < 0));int fq = (q.y <
     0 || (q.y == 0 && q.x < 0));
if (fp != fq)return fp == 0;if (cross(p, q))return cross(p,
     q) > 0;return cross(p, rhs.b - a) < 0;}PT
     line_line_intersection(PT a, PT b, PT c, PT d)
{b = b - a;d = c - d;c = c - a;return a + b * cross(c, d) /
     cross(b, d);}
PT intersection(const HP &v){return line_line_intersection(a
     , b, v.a, v.b);}};
int check(HP a, HP b, HP c){return cross(a.b - a.a, b.
     intersection(c) - a.a) > -eps; //-eps to include
     polygons of zero area (straight lines, points)}
// consider half-plane of counter-clockwise side of each
     line
// if lines are not bounded add infinity rectangle
// returns a convex polygon, a point can occur multiple
     times though
// complexity: O(n log(n))
vector<PT> half_plane_intersection(vector<HP> h){sort(h.
     begin(), h.end());vector<HP> tmp;
for (int i = 0; i < h.size(); i++){if (!i || cross(h[i].b -
     h[i].a, h[i - 1].b - h[i - 1].a)){tmp.push_back(h[i])
     ;}}
h = tmp;vector<HP> q(h.size() + 10);int qh = 0, qe = 0;
for (int i = 0; i < h.size(); i++){
while (qe - qh > 1 && !check(h[i], q[qe - 2], q[qe - 1]))qe
     --;
while (qe - qh > 1 && !check(h[i], q[qh], q[qh + 1]))qh++;q[
     qe++] = h[i];}
while (qe - qh > 2 && !check(q[qh], q[qe - 2], q[qe - 1]))qe
     --;
while (qe - qh > 2 && !check(q[qe - 1], q[qh], q[qh + 1]))qh
     ++;
vector<HP> res;
for (int i = qh; i < qe; i++)res.push_back(q[i]);vector<PT>
     hull;
if (res.size() > 2){for (int i = 0; i < res.size(); i++){
     hull.push_back(res[i].intersection(res[(i + 1) % ((int)
     res.size())]));}}
return hull;}
// rotate the polygon such that the (bottom, left)-most
     point is at the first position
void reorder_polygon(vector<PT> &p){int pos = 0;for (int i =
      1; i < p.size(); i++){if (p[i].y < p[pos].y || (sign(p
     [i].y - p[pos].y) == 0 && p[i].x < p[pos].x))pos = i;}
     rotate(p.begin(), p.begin() + pos, p.end());}
// a and b are convex polygons
// returns a convex hull of their minkowski sum
// min(a.size(), b.size()) >= 2
// https://cp-algorithms.com/geometry/minkowski.html
vector<PT> minkowski_sum(vector<PT> a, vector<PT> b){
reorder_polygon(a);reorder_polygon(b);
int n = a.size(), m = b.size();int i = 0, j = 0;a.push_back(
     a[0]);a.push_back(a[1]);b.push_back(b[0]);b.push_back(b
     [1]);vector<PT> c;
while (i < n || j < m){c.push_back(a[i] + b[j]);double p =
     cross(a[i + 1] - a[i], b[j + 1] - b[j]);if (sign(p) >=
     0)++i;if (sign(p) <= 0)++j;}
return c;}
// returns the area of the intersection of the circle with
     center c and radius r
// and the triangle formed by the points c, a, b
double _triangle_circle_intersection(PT c, double r, PT a,
     PT b){
double sd1 = dist2(c, a), sd2 = dist2(c, b);
if (sd1 > sd2)swap(a, b), swap(sd1, sd2);double sd = dist2(a
     , b);double d1 = sqrtl(sd1), d2 = sqrtl(sd2), d = sqrt(
     sd);double x = abs(sd2 - sd - sd1) / (2 * d);double h =
      sqrtl(sd1 - x * x);
if (r >= d2)return h * d / 2;double area = 0;
if (sd + sd1 < sd2){if (r < d1)area = r * r * (acos(h / d2)
     - acos(h / d1)) / 2;else{area = r * r * (acos(h / d2) -
      acos(h / r)) / 2;double y = sqrtl(r * r - h * h);area
```

```
    += h * (y - x) / 2;}}
else{if (r < h)area = r * r * (acos(h / d2) + acos(h / d1))
    / 2;else{area += r * r * (acos(h / d2) - acos(h / r)) /
     2;double y = sqrtl(r * r - h * h);area += h * y / 2;if
     (r < d1){area += r * r * (acos(h / d1) - acos(h / r))
    / 2;area += h * y / 2;}elsearea += h * x / 2;}}
return area;}
// intersection between a simple polygon and a circle
double polygon_circle_intersection(vector<PT> &v, PT p,
    double r){int n = v.size();double ans = 0.00;PT org =
    {0, 0};
for (int i = 0; i < n; i++){int x = orientation(p, v[i], v[(
    i + 1) % n]);if (x == 0)continue;double area =
    _triangle_circle_intersection(org, r, v[i] - p, v[(i +
    1) % n] - p);if (x < 0)ans -= area;elseans += area;}
return abs(ans);}
// find a circle of radius r that contains as many points as
     possible
// O(n^2 log n);
double maximum_circle_cover(vector<PT> p, double r, circle &
    c){int n = p.size();int ans = 0;int id = 0;double th =
    0;
for (int i = 0; i < n; ++i){
// maximum circle cover when the circle goes through this
    point
vector<pair<double, int>> events = {{-PI, +1}, {PI, -1}};
for (int j = 0; j < n; ++j){if (j == i)continue;double d =
    dist(p[i], p[j]);if (d > r * 2)continue;
double dir = (p[j] - p[i]).arg();double ang = acos(d / 2 / r
    );double st = dir - ang, ed = dir + ang;
if (st > PI)st -= PI * 2;if (st <= -PI)st += PI * 2;if (ed >
     PI)ed -= PI * 2;if (ed <= -PI)ed += PI * 2;
events.push_back({st - eps, +1}); // take care of precisions
    !
events.push_back({ed, -1});if (st > ed){events.push_back({-
    PI, +1});events.push_back({+PI, -1});}}sort(events.
    begin(), events.end());int cnt = 0;for (auto &&e :
    events){cnt += e.second;if (cnt > ans){ans = cnt;id = i
    ;th = e.first;}}}PT w = PT(p[id].x + r * cos(th), p[id
    ].y + r * sin(th));c = circle(w, r); // best_circle
return ans;}
// radius of the maximum inscribed circle in a convex
    polygon
double maximum_inscribed_circle(vector<PT> p){int n = p.size
    ();if (n <= 2)return 0;double l = 0, r = 20000;
while (r - l > eps){double mid = (l + r) * 0.5;vector<HP> h;
    const int L = 1e9;h.push_back(HP(PT(-L, -L), PT(L, -L))
    );h.push_back(HP(PT(L, -L), PT(L, L)));h.push_back(HP(
    PT(L, L), PT(-L, L)));h.push_back(HP(PT(-L, L), PT(-L,
    -L)));for (int i = 0; i < n; i++){PT z = (p[(i + 1) % n
    ] - p[i]).perp();z = z.truncate(mid);PT y = p[i] + z, q
     = p[(i + 1) % n] + z;h.push_back(HP(p[i] + z, p[(i +
    1) % n] + z));}vector<PT> nw = half_plane_intersection(
    h);if (!nw.empty())l = mid;elser = mid;}
return l;}
// ear decomposition, O(n^3) but faster
vector<vector<PT>> triangulate(vector<PT> p){
vector<vector<PT>> v;
while (p.size() >= 3){
for (int i = 0, n = p.size(); i < n; i++){int pre = i == 0 ?
     n - 1 : i - 1;;int nxt = i == n - 1 ? 0 : i + 1;int
    ori = orientation(p[i], p[pre], p[nxt]);if (ori < 0){
    int ok = 1;for (int j = 0; j < n; j++){if (j == i || j
    == pre || j == nxt)continue;if (is_point_in_triangle(p[
    i], p[pre], p[nxt], p[j]) < 1){ok = 0;break;}}if (ok){v
    .push_back({p[pre], p[i], p[nxt]});p.erase(p.begin() +
    i);break;}}}}
return v;}

struct star{
int n;   // number of sides of the star
double r; // radius of the circumcircle
star(int _n, double _r){n = _n;r = _r;}
double area(){double theta = PI / n;double s = 2 * r * sin(
    theta);double R = 0.5 * s / tan(theta);double a = 0.5 *
     n * s * R;double a2 = 0.25 * s * s / tan(1.5 * theta);
    return a - n * a2;}};

// given a list of lengths of the sides of a polygon in
    counterclockwise order
// returns the maximum area of a non-degenerate polygon that
     can be formed using those lengths
double get_maximum_polygon_area_for_given_lengths(vector<
    double> v){
if (v.size() < 3){return 0;}
int m = 0;double sum = 0;
for (int i = 0; i < v.size(); i++){if (v[i] > v[m]){m = i;}
    sum += v[i];}
if (sign(v[m] - (sum - v[m])) >= 0){return 0; // no non-
    degenerate polygon is possible}
// the polygon should be a circular polygon
// that is all points are on the circumference of a circle
double l = v[m] / 2, r = 1e6; // fix it correctlyint it =
    60;
auto ang = [](double x, double r) { // x = length of the
    chord, r = radius of the circle
return 2 * asin((x / 2) / r);};
auto calc = [=](double r){double sum = 0;for (auto x : v){
    sum += ang(x, r);}return sum; };
// compute the radius of the circle
while (it--){double mid = (l + r) / 2;if (calc(mid) <= 2 *
    PI){r = mid;}else{l = mid;}}
if (calc(r) <= 2 * PI - eps){ // the center of the circle is
     outside the polygon
auto calc2 = [&](double r){double sum = 0;for (int i = 0; i
    < v.size(); i++){double x = v[i];double th = ang(x, r);
    if (i != m){sum += th;}else{sum += 2 * PI - th;}}return
     sum;};l = v[m] / 2;r = 1e6;it = 60;while (it--){
double mid = (l + r) / 2;if (calc2(mid) > 2 * PI){r = mid;}
    else{l = mid;}}
auto get_area = [=](double r){double ans = 0;
for (int i = 0; i < v.size(); i++){double x = v[i];double
    area = r * r * sin(ang(x, r)) / 2;if (i != m){ans +=
    area;}else{ans -= area;}}return ans;};return get_area(r
    );}else{ // the center of the circle is inside the
    polygon
auto get_area = [=](double r){double ans = 0;for (auto x : v
    ){ans += r * r * sin(ang(x, r)) / 2;}return ans;};
    return get_area(r);}}
```

3.2 Grundy

```
int dp[N];//two players playing with unequal partion
int Grundy(int n){
    if (n <= 2)
        return 0;
    if (dp[n] != -1)
        return dp[n];
    vector<int>vis(1005, 0);
    for (int i = 1 ; i < n ; i++){
        if (n - i != i){
            vis[(Grundy(n - i)^Grundy(i))] = 1;
        }
    }
    int p = 0;
    while(vis[p]) p++;
    return dp[n] = p;
}
```

3.3 build

```
//windows
{
"shell_cmd": "g++.exe -std=c++17 \"$file\" -o \"
    $file_base_name\".exe && \"$file_base_name.exe\"<\"D://
    c programes/in.txt\">\"D://c programes/out.txt\"",
"shell":true,
```

Murad Linux: { "cmd": ["bash", "-c", "g++ \"$file\" -o \"$file_base_name.exec\" && ./$file_base_name.exec < input.txt > output.txt"] }

```
"working_dir":"$file_path",
"selector":"source.c, source.cpp, source.c++"
}
//linux
{
"cmd" : ["g++ -std=c++17 \"$file_name\" && timeout 10s ./a.
    out <~/Code/input.txt | head -n 2000000 | head -c
    50000000 >~/Code/output.txt"],
"selector" : "source.c,source.cpp,source.c++",
"shell": true,
"working_dir" : "$file_path"
}
```

3.4 formula

a|b = (a^b)|(a&b)

a^(a&b) = (a|b)^b

b^(a&b) = (a|b)^a

(a&b)^(a|b) = a^b

Subtraction:

a-b = (a^(a&b)) - ((a|b)^a)

a-b = ((a|b)^b) - ((a|b)^a)

a-b = (a^(a&b)) - (b^(a&b))

a-b = ((a|b)^b) - (b^(a&b))

Addition:

a+b = (a^b)+2*(a&b)

a+b = (a|b)+(a&b)

3.5 pragma

```
// #pragma GCC optimize("O3,unroll-loops,Ofast")
// #pragma GCC target("avx2")
```

3.6 template

```
1. nCr = nCn-r
2. nCr = n-1Cr-1 + n-1Cr
2. (1+x)^n = sum of r[0:n] (nCr)*x^r
3. sum of r[0:n] (nCr) = 2^n
4. 2nCn =sum of r[0:n] (nCr)^2

#define fastio {ios_base::sync_with_stdio(false);cin.tie(0);
    cout.tie(0);}
```

```
std::vector<int> vec1; vec1.assign(5, 10); // Fills vec1
    with 5 elements, all 10
std::vector<int> vec2(5, 0); std::fill(vec2.begin(), vec2.
    end(), 20); // Fills vec2 with 20
std::vector<int> vec3(3, 0); vec3.resize(5, 100); // Resizes
     vec3 to 5 elements, new elements are 100
std::vector<int> vec4; vec4.emplace_back(42); // Adds 42 to
    the end of vec4

bitset<size> variable_name;
b1; b1.set(3); // Set the 3rd bit to 1
b2(255); b2.reset(7); // Reset the 7th bit to 0
b3(5); b3.flip(2); // Flip the 2nd bit
b4(42); b4.count(); // Count the number of set bits
b5(5); b5.test(2); // Check if the 2nd bit is set
b6(0); b6.any(); // Check if any bit is set
b7(0); b7.none(); // Check if no bit is set
b8(255); b8.all(); // Check if all bits are set
b9(1); b9 <<= 2; // Left shift by 2
b10(4); b10 >>= 1; // Right shift by 1

&lt; and &gt; instead of < and >
```

4 Graph Theory

4.1 Articulation bridge

```
vector<int>node[10003];
int lowtime[10003],intime[10003],vis[10003];
vector<pair<int,int>>edge;
int timer;
void dfs(int p, int parent){
      intime[p]=lowtime[p]=timer;
      timer++;
      vis[p]=1;
      for(int child:node[p]){
          if(child==parent)
          continue;
          if(vis[child]){
             lowtime[p]=min(lowtime[p], intime[child]); //
                 node - child is a back edge
          }
          else{
              // node - child is a forward edge
              dfs(child,p);
              if(lowtime[child] > intime[p])
               edge.push_back({p , child});
              lowtime[p] = min(lowtime[p] , lowtime[child]);
          }
      }
}
```

4.2 Articulation point

```
vector<int>node[10003];
int lowtime[10003],intime[10003],vis[10003];
set<int>cut_vertex;
int timer;
void IS_CUTPOINT(int x){
    cut_vertex.insert(x);
}
void dfs(int p, int parent){
      intime[p]=lowtime[p]=timer;
      timer++;
      int children=0;
      vis[p]=1;
      for(int child:node[p]){
          if(child==parent)
          continue;
          if(vis[child]){
             lowtime[p]=min(lowtime[p], intime[child]); //
                 node - child is a back edge
          }
          else{
              // node - child is a forward edge
              dfs(child,p);
              if(lowtime[child] >= intime[p] && parent!=-1)
               IS_CUTPOINT(p);
              lowtime[p] = min(lowtime[p] , lowtime[child]);
              children++;
          }
      }
      if(parent == -1 && children>1)//for root
         IS_CUTPOINT(p);
}
```

4.3 Bellman ford

```
struct edge{
   int a,b,cost;
};
vector<edge>node;
int n,m,root_node;
void solve(){
    vector<int>dis(n,inf);
```

```
    dis[root_node]=0;
    for(int i=0;i<n-1;i++){
       for(int j=0;j<m;j++){
          if(dis[node[j].a]<inf){
             dis[node[j].b]=min(dis[node[j].b], dis[node[j].a
                  ]+node[j].cost);
          }
       }
    }
}
```

4.4 Dijsktra

```
void dijkstra_algo(ll n, vector<vector<ll>>& adj, vector<
     vector<ll>>& weight,
    vector<ll>& cost, vector<ll>& parent, ll s, ll d) {
    priority_queue<pl, vector<pl>, greater<pl>> pq;
    pq.push(mk(0, s));
    while (!pq.empty()) {
        ll node = pq.top().se;
        ll current_cost = pq.top().fi;
        pq.pop();
        if (cost[node] < current_cost) continue; // Node is
             already explored by the shortest path
        for (ll i = 0; i < adj[node].size(); i++) {
            ll temp = adj[node][i];
            if (current_cost + weight[node][i] < cost[temp])
                 {
                cost[temp] = current_cost + weight[node][i];
                parent[temp] = node;
                pq.push(mk(cost[temp], temp));
            }
        }
    }
}
vector<ll> path;
for (ll v = d; v != -1; v = parent[v]){
    path.push_back(v);
}
reverse(path.begin(), path.end());
```

5 Number theory

5.1 bigmod

```
ll bigmod(ll a, ll b, ll n) {
    ll res = 1; if (b == 0) return 1;
    a = a % n; if (a == 0)return 0;
    while (b > 0) {
        if (b % 2)res = (res * a) % n;
        b = b / 2; a = (a * a) % n;
    }
    return res;
}
```

5.2 extended euclid

```
int x, y;
int gcdExtended(int a, int b, int *x, int *y) {
   if (b == 0) {
      *x = 1; *y = 0; return a;
   }
   int x1, y1;
   int gcd = gcdExtended(b, a % b, &x1, &y1);
   *x = y1; *y = x1 - y1 * (a / b); return gcd;
}
```

5.3 linear diphantine equation

```
ll extended_euclid(ll a, ll b, ll &x, ll &y) { ll xx = y =
     0; ll yy = x = 1; while (b) { ll q = a / b; ll t = b; b
      = a % b; a = t; t = xx; xx = x - q * xx; x = t; t = yy
     ; yy = y - q * yy; y = t; } return a; } // Extended
     Euclidean Algorithm to find gcd and coefficients x, y.
bool find_any_solution (ll a, ll b, ll c, ll &x0, ll &y0, ll
      &g) { if (a == 0 and b == 0) { if (c) return false; x0
      = y0 = g = 0; return true; } g = extended_euclid (abs(
     a), abs(b), x0, y0); if (c % g != 0) return false; x0
     *= c / g; y0 *= c / g; if (a < 0) x0 *= -1; if (b < 0)
     y0 *= -1; return true; } // Solves for x0, y0 for the
     equation a*x + b*y = c, returns true if possible, false
      otherwise.
void shift_solution(ll &x, ll &y, ll a, ll b, ll cnt) { x +=
      cnt * b; y -= cnt * a; } // Shifts the solution by cnt
      to ensure x, y remain valid solutions for the equation
     .
ll find_all_solutions(ll a, ll b, ll c, ll minx, ll maxx, ll
      miny, ll maxy) { ll x, y, g; if (find_any_solution(a,
     b, c, x, y, g) == 0) return 0; if (a == 0 and b == 0) {
      assert(c == 0); return 1LL * (maxx - minx + 1) * (maxy
      - miny + 1); } if (a == 0) { return (maxx - minx + 1)
     * (miny <= c / b and c / b <= maxy); } if (b == 0) {
     return (maxy - miny + 1) * (minx <= c / a and c / a <=
     maxx); } a /= g, b /= g; ll sign_a = a > 0 ? +1 : -1;
     ll sign_b = b > 0 ? +1 : -1; shift_solution(x, y, a, b,
      (minx - x) / b); if (x < minx) shift_solution(x, y, a,
      b, sign_b); if (x > maxx) return 0; ll lx1 = x;
     shift_solution(x, y, a, b, (maxx - x) / b); if (x >
     maxx) shift_solution(x, y, a, b, -sign_b); ll rx1 = x;
     shift_solution(x, y, a, b, -(miny - y) / a); if (y <
     miny) shift_solution(x, y, a, b, -sign_a); if (y > maxy
     ) return 0; ll lx2 = x; shift_solution(x, y, a, b, -(
     maxy - y) / a); if (y > maxy) shift_solution(x, y, a, b
     , sign_a); ll rx2 = x; if (lx2 > rx2) swap(lx2, rx2);
     ll lx = max(lx1, lx2); ll rx = min(rx1, rx2); if (lx >
     rx) return 0; return (rx - lx) / abs(b) + 1; } // Finds
      the number of integer solutions within the given
     bounds for x and y.
int32_t main() { ios_base::sync_with_stdio(0); cin.tie(0);
     int t, cs = 0; cin >> t; while (t--) { ll a, b, c, x1,
     x2, y1, y2; cin >> a >> b >> c >> x1 >> x2 >> y1 >> y2;
      cout << "Case " << ++cs << ": " << find_all_solutions(
     a, b, -c, x1, x2, y1, y2) << '\n'; } return 0; } //
     Main driver to process multiple test cases.
```

5.4 linear sieve

```
void lin_sieve() { for (ll i = 2; i < N; i++) if (!isPrime[i
     ]) p.push_back(i), for (ll j : p) if (i * j >= N || !(
     isPrime[i * j] = 1) && i % j == 0) break; }
```

5.5 mat expo

```
const int mod = 1e9 + 7;
// const int N = 5e5 + 6;
// No of terms in the Recurrence Relation.
const int N = 4;
const long long M = 1000000007;
// Multiplies two matrices A and B and stores the result in
     A.
void multiply(long long A[N][N], long long B[N][N]) { long
     long R[N][N] = {}; for (int i = 0; i < N; i++) for (int
      j = 0; j < N; j++) for (int k = 0; k < N; k++) R[i][j]
      = (R[i][j] + A[i][k] * B[k][j]) % M; for (int i = 0; i
      < N; i++) for (int j = 0; j < N; j++) A[i][j] = R[i][j
     ]; }
// Raise matrix A to the power of n in O(log n).
void power_matrix(long long A[N][N], ll n) { long long B[N][
     N]; for (int i = 0; i < N; i++) for (int j = 0; j < N;
```

```
    j++) B[i][j] = A[i][j]; for (n--; n > 0; n >>= 1,
    multiply(B, B)) if (n & 1) multiply(A, B); }
// A = Coefficient Matrix, B = Base Matrix.
// It returns the nth term of the recurrence relation formed
     from A and B in O(log n).
long long solve_recurrence(long long A[N][N], long long B[N
    ][1], ll n) { if (n < N) return B[N - 1 - n][0];
    power_matrix(A, n - N + 1); long long result = 0; for (
    int i = 0; i < N; i++) result = (result + A[0][i] * B[i
    ][0]) % M; return result; }
void Solve() {
  /*
      The recurrence relation used here is: -
      R(n) = 2 * R(n-1) + R(n-2) + 3 * R(n-3) + 3.
      Base Cases: R(0) = 1, R(1) = 2, R(2) = 3.
   */
  // Forming the Coefficient Matrix
  long long A[N][N] = {{3, 2, 1, 1}, {1, 0, 0, 0}, {0, 1, 0,
       0}, {0, 0, 0, 1}};
  // Forming the Base Matrix
  long long B[N][1] = {{3}, {2}, {1}, {3}};
  ll n;
  cin >> n;
  if (n <= 2) {
    cout << n + 1 << endl;
    return;
  }
  long long R_n = solve_recurrence(A, B, n + 1);
  // cout << "R_" << n << " = " << R_n;
  cout << R_n << endl;
}

int main() {
  Solve();
  return 0;
}
```

5.6 ncr for mod

```
ll fact[200008];
ll bigmod(ll b, ll p) {
    // implement
}
ll ncr(ll n, ll r) {
 if(n<r)
  return 0;
   return fact[n] * bigmod(fact[r] * fact[n-r] % mod, mod -
        2) % mod;
}
void Fact(){
    fact[0] = 1;
    for(int i=1; i<=200002; i++) fact[i] = fact[i-1] * i %
        mod;
}
```

5.7 phi

```
// for n <= 1e6 by nloglogn
void phi(int n) { vector<int> phi(n + 1); for (int i = 0; i
    <= n; i++) phi[i] = i; for (int i = 2; i <= n; i++) if
    (phi[i] == i) for (int j = i; j <= n; j += i) phi[j] -=
     phi[j] / i; }
// for single number by sqrt(n)
int phi(int n) { int res = n; for (int i = 2; i * i <= n; i
    ++) if (n % i == 0) { while (n % i == 0) n /= i; res -=
     res / i; } if (n > 1) res -= res / n; return res; }
```

5.8 pollard rho

```
using ll = long long;
namespace PollardRho {
  mt19937 rnd(chrono::steady_clock::now().time_since_epoch()
      .count());
  const int P = 1e6 + 9;
  ll seq[P];
  int primes[P], spf[P];
  inline ll add_mod(ll x, ll y, ll m) { return (x += y) < m
      ? x : x - m; }
  inline ll mul_mod(ll x, ll y, ll m) { return __int128(x) *
       y % m; }
  inline ll pow_mod(ll x, ll n, ll m) { ll res = 1 % m; for
      (; n; n >>= 1) { if (n & 1) res = mul_mod(res, x, m);
      x = mul_mod(x, x, m); } return res; }
  inline bool miller_rabin(ll n) { if (n <= 2 || (n & 1 ^ 1)
      ) return (n == 2); if (n < P) return spf[n] == n; ll c
      , d, s = 0, r = n - 1; for (; !(r & 1); r >>= 1, s++)
      {} for (int i = 0; primes[i] < n && primes[i] < 32; i
      ++) { c = pow_mod(primes[i], r, n); for (int j = 0; j
      < s; j++) { d = mul_mod(c, c, n); if (d == 1 && c != 1
       && c != (n - 1)) return false; c = d; } if (c != 1)
      return false; } return true; }
  void init() { int cnt = 0; for (int i = 2; i < P; i++) {
      if (!spf[i]) primes[cnt++] = spf[i] = i; for (int j =
      0, k; (k = i * primes[j]) < P; j++) { spf[k] = primes[
      j]; if (spf[i] == spf[k]) break; } } }
  ll pollard_rho(ll n) { while (1) { ll x = rnd() % n, y = x
      , c = rnd() % n, u = 1, v, t = 0; ll *px = seq, *py =
      seq; while (1) { *py++ = y = add_mod(mul_mod(y, y, n),
       c, n); *py++ = y = add_mod(mul_mod(y, y, n), c, n);
      if ((x = *px++) == y) break; v = u; u = mul_mod(u, abs
      (y - x), n); if (!u) return __gcd(v, n); if (++t ==
      32) { t = 0; if ((u = __gcd(u, n)) > 1 && u < n)
      return u; } } if (t && (u = __gcd(u, n)) > 1 && u < n)
       return u; } }
  vector<ll> factorize(ll n) { if (n == 1) return vector<ll
      >(); if (miller_rabin(n)) return vector<ll>{n}; vector
      <ll> v, w; while (n > 1 && n < P) { v.push_back(spf[n
      ]); n /= spf[n]; } if (n >= P) { ll x = pollard_rho(n)
      ; v = factorize(x); w = factorize(n / x); v.insert(v.
      end(), w.begin(), w.end()); } return v; }
}
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  PollardRho::init();
  int t; cin >> t;
  while (t--) {
    ll n; cin >> n;
    auto f = PollardRho::factorize(n);
    sort(f.begin(), f.end());
    cout << f.size() << ' ';
    for (auto x : f) cout << x << ' ';
    cout << '\n';
  }
  return 0;
}
// https://judge.yosupo.jp/problem/factorize
```

5.9 power tower

```
const int N = 1e5 + 9;
using ll = long long;
map<ll, ll> mp;
ll phi(ll n) { if (mp.count(n)) return mp[n]; ll ans = n, m
    = n; for (ll i = 2; i * i <= m; i++) { if (m % i == 0)
    { while (m % i == 0) m /= i; ans = ans / i * (i - 1); }
     } if (m > 1) ans = ans / m * (m - 1); return mp[n] =
    ans; }
inline ll MOD(ll x, ll m) { if (x < m) return x; return x %
    m + m; }
ll power(ll n, ll k, ll mod) { ll ans = MOD(1, mod); while (
    k) { if (k & 1) ans = MOD(ans * n, mod); n = MOD(n * n,
     mod); k >>= 1; } return ans; }
int a[N];
```

```
ll yo(ll l, ll r, ll m) { if (l == r) return MOD(a[l], m);
    if (m == 1) return 1; return power(a[l], yo(l + 1, r,
    phi(m)), m); }
int32_t main() { ios_base::sync_with_stdio(0); cin.tie(0);
    int n, m; cin >> n >> m; for (int i = 1; i <= n; i++) {
     cin >> a[i]; } int q; cin >> q; while (q--) { int l, r
    ; cin >> l >> r; cout << yo(l, r, m) % m << '\n'; }
    return 0; }
// https://codeforces.com/contest/906/problem/D
```

5.10 sieve all

```
vector<bool> prime(N, true); vector<int> vec; void seive() {
     prime[0] = false; prime[1] = false; for (int i = 2; i
    * i < N; i++) { if (prime[i]) { for (int j = i * i; j <
     N; j += i) { prime[j] = false; } } } for (int i = 2; i
     < N; i++) if (prime[i]) vec.push_back(i); } void pro()
     { int n; cin >> n; int ans = 1; for (auto it : vec) {
    if (it * it > n) break; if (n % it == 0) { int cnt = 1;
     while (n % it == 0) { n /= it; cnt++; } ans *= cnt; }
    } if (n > 1) ans *= 2; cout << ans - 1 << endl; } void
    pro() { int n, m; cin >> n >> m; bool ara[m - n + 1];
    memset(ara, true, sizeof(ara)); for (auto it : sve) {
    if (it * it > m) break; int fmpl = (n + it - 1) / it;
    fmpl *= it; int strt = max(fmpl, it * it); for (int j =
     strt; j <= m; j += it) { ara[j - n] = false; } } if (n
     == 1) ara[0] = false; for (int i = n; i <= m; i++) if
    (ara[i - n]) cout << i << endl; }
```

5.11 sieve

```
#define sz 100000010
bool siv[sz];
vector<long long >primes;
void seive(){
    long long limit = sqrt(sz+1);
    for (long long i=3; i<=limit; i+=2)
        if(!siv[i])
            for (long j = i*i; j<sz; j+=i+i)
                siv[j] = 1;
    primes.push_back(2);
    for (long long i=3; i<sz; i+=2)
        if(!siv[i]) primes.push_back(i);
}
```

5.12 totient

```
#include<bits/stdc++.h> using namespace std; const double pi
     = 2 * acos(0.0); const int N = 5000006; const int INF
    = INT_MAX; const int mod = 1000000007; vector<int> phi(
    N, 0); void totient_seive() { for (int i = 1; i < N; i
    ++) phi[i] = i; for (int i = 2; i < N; i++) { if (phi[i
    ] == i) { for (int j = i; j < N; j += i) { phi[j] = (
    phi[j] - (phi[j] / i)); } } } } const int N =
    1000009; const int INF = INT_MAX; const int mod =
    1000000007; int sqrt_phi(int n) { int ans = n; for (int
     i = 2; i * i <= n; i++) { if (n % i == 0) { while (n %
     i == 0) n /= i; ans -= (ans / i); } } if (n > 1) ans
    -= (ans / n); return ans; } const int N = 10000007;
    const int INF = INT_MAX; const int mod = 1000000007;
    vector<int> phi(N, 0); void calcphi() { phi[0] = 0; phi
    [1] = 1; for (int i = 2; i < N; i++) phi[i] = i - 1;
    for (int i = 2; i < N; i++) { for (int j = 2 * i; j < N
    ; j += i) { phi[j] -= phi[i]; } } }
```

6 String

6.1 AhoCorasick

```
struct AC {
    int N, P; const int A = 26;
    vector<vector<int>> next; vector<int> link, out_link;
         vector<vector<int>> out;
    AC() : N(0), P(0) { node(); }
    int node() { next.emplace_back(A, 0); link.emplace_back
         (0); out_link.emplace_back(0); out.emplace_back(0);
         return N++; }
    inline int get(char c) { return c - 'a'; }
    int add_pattern(const string T) { int u = 0; for (auto c
         : T) { if (!next[u][get(c)]) next[u][get(c)] = node
         (); u = next[u][get(c)]; } out[u].push_back(P);
         return P++; }
    void compute() { queue<int> q; for (q.push(0); !q.empty()
         ;) { int u = q.front(); q.pop(); for (int c = 0; c <
          A; ++c) { int v = next[u][c]; if (!v) next[u][c] =
         next[link[u]][c]; else { link[v] = u ? next[link[u
         ]][c] : 0; out_link[v] = out[link[v]].empty() ?
         out_link[link[v]] : link[v]; q.push(v); } } } }
    int advance(int u, char c) { while (u && !next[u][get(c)
         ]) u = link[u]; u = next[u][get(c)]; return u; }
    void match(string text) { int u = 0; for (int i = 0; i <
         text.size(); ++i) { u = advance(u, text[i]); for (
         int v = u; v; v = out_link[v]) for (int pattern_idx
         : out[v]) cout << "Pattern " << pattern_idx << "
         found at index " << i << endl; } }
};
```

6.2 Hashing

```
const int N = 1e6 + 9;

int power(long long n, long long k, const int mod) {
  int ans = 1 % mod;
  n %= mod;
  if (n < 0) n += mod;
  while (k) {
    if (k & 1) ans = (long long) ans * n % mod;
    n = (long long) n * n % mod;
    k >>= 1;
  }
  return ans;
}
const int MOD1 = 127657753, MOD2 = 987654319;
const int p1 = 151, p2 = 239;
int ip1, ip2;
pair<int, int> pw[N], ipw[N];
void prec() {
  pw[0] = {1, 1};
  for (int i = 1; i < N; i++) {
    pw[i].first = 1LL * pw[i - 1].first * p1 % MOD1;
    pw[i].second = 1LL * pw[i - 1].second * p2 % MOD2;
  }
  ip1 = power(p1, MOD1 - 2, MOD1);
  ip2 = power(p2, MOD2 - 2, MOD2);
  ipw[0] = {1, 1};
  for (int i = 1; i < N; i++) {
    ipw[i].first = 1LL * ipw[i - 1].first * ip1 % MOD1;
    ipw[i].second = 1LL * ipw[i - 1].second * ip2 % MOD2;
  }
}
struct Hashing {
  int n;
  string s; // 0 - indexed
  vector<pair<int, int>> hs; // 1 - indexed
  Hashing() {}
  Hashing(string _s) {
    n = _s.size();
    s = _s;
    hs.emplace_back(0, 0);
    for (int i = 0; i < n; i++) {
      pair<int, int> p;
```

```
      p.first = (hs[i].first + 1LL * pw[i].first * s[i] %
          MOD1) % MOD1;
      p.second = (hs[i].second + 1LL * pw[i].second * s[i] %
          MOD2) % MOD2;
      hs.push_back(p);
    }
  }
  pair<int, int> get_hash(int l, int r) { // 1 - indexed
    assert(1 <= l && l <= r && r <= n);
    pair<int, int> ans;
    ans.first = (hs[r].first - hs[l - 1].first + MOD1) * 1LL
        * ipw[l - 1].first % MOD1;
    ans.second = (hs[r].second - hs[l - 1].second + MOD2) * 1
        LL * ipw[l - 1].second % MOD2;
    return ans;
  }
  pair<int, int> get_hash() {
    return get_hash(1, n);
  }
};
```

6.3 LPS

```
vector<int> lps_function(string P) { vector<int> pi(P.size()
    ); pi[0] = 0; int q = 0; for (int i = 1; i < (int) P.
    size(); ++i) { while (q > 0 and P[q] != P[i]) q = pi[q
    - 1]; if (P[q] == P[i]) ++q; pi[i] = q; } return pi; }
int f(int m, char c, const string& s, const vector<int>& pi)
    { while (m > 0 and s[m] != c) m = pi[m - 1]; if (s[m]
    == c) ++m; return m; }
```

6.4 Manachar

```
struct Manacher { vector<int> p[2]; Manacher(string s) { int
    n = s.size(); p[0].resize(n + 1); p[1].resize(n); for
    (int z = 0; z < 2; z++) for (int i = 0, l = 0, r = 0; i
    < n; i++) { int t = r - i + !z; if (i < r) p[z][i] =
    min(t, p[z][l + t]); int L = i - p[z][i], R = i + p[z][
    i] - !z; while (L >= 1 && R + 1 < n && s[L - 1] == s[R
    + 1]) p[z][i]++, L--, R++; if (R > r) l = L, r = R; } }
    } bool is_palindrome(int l, int r) { int mid = (l + r
    + 1) / 2, len = r - l + 1; return 2 * p[len % 2][mid] +
    len % 2 >= len; } };
```

6.5 MaximumLexicographicRotation

```
int maxLexi(string s) { int len = s.size(), n = 2 * len, l =
    0, j = 1, k = 0; while (l + k < n && j + k < n) { int
    a = (l + k >= len) ? s[l + k - len] : s[l + k], b = (j
    + k >= len) ? s[j + k - len] : s[j + k]; if (a == b) k
    ++; else if (a < b) { l = l + k + 1; if (l <= j) l = j
    + 1; k = 0; } else { j = j + k + 1; if (j <= l) j = l +
    1; k = 0; } } return min(l, j); }
```

6.6 MinimumLexicographicRotaion

```
int minLexi(string s) { int len = s.size(), n = 2 * len, l =
    0, j = 1, k = 0; while (l + k < n && j + k < n) { int
    a = (l + k >= len) ? s[l + k - len] : s[l + k], b = (j
    + k >= len) ? s[j + k - len] : s[j + k]; if (a == b) k
    ++; else if (a > b) { l = l + k + 1; if (l <= j) l = j
    + 1; k = 0; } else { j = j + k + 1; if (j <= l) j = l +
    1; k = 0; } } return min(l, j); }
```

6.7 PersistentBitwiseTrie

```
double pi = 3.14159265359; const int LG = 18, p1 = 31, p2 =
    53; unsigned long long pow1[32], pow2[32]; void prec()
    { pow1[0] = 1; pow2[0] = 1; for (int i = 1; i <= 31; i
    ++) { pow1[i] = p1 * pow1[i - 1]; pow2[i] = p2 * pow2[i
    - 1]; } }
struct Node { unsigned long long hsh = 0; Node *left = 0, *
    right = 0; bool end = false; }; Node* upd(Node *v, int
    x, int i = LG) { Node *res = new Node(); if (i == -1) {
    res->hsh = ((int(1e9) + 345) ^ x) * (3 * x + 654); res
    ->end = true; return res; } if ((x >> i) & 1) { res->
    left = v ? v->left : 0; res->right = upd(v ? v->right :
    0, x, i - 1); } else { res->right = v ? v->right : 0;
    res->left = upd(v ? v->left : 0, x, i - 1); } unsigned
    long long a = (res->left ? res->left->hsh * pow1[i + 1]
    : 0); unsigned long long b = (res->right ? res->right
    ->hsh * pow2[i + 1] : 0); res->hsh = a + b; return res;
    } bool find(Node *v, int x, int i = LG) { if (v == 0)
    return false; if (i == -1) return v->end; if ((x >> i)
    & 1) return find(v->right, x, i - 1); return find(v->
    left, x, i - 1); } Node* del(Node *v, int x, int i = LG
    ) { if (i == -1) return 0; Node* res = new Node(); res
    ->left = v->left; res->right = v->right; res->end = v->
    end; res->hsh = v->hsh; if ((x >> i) & 1) res->right =
    del(res->right, x, i - 1); else res->left = del(res->
    left, x, i - 1); unsigned long long a = (res->left ?
    res->left->hsh * pow1[i + 1] : 0); unsigned long long b
    = (res->right ? res->right->hsh * pow2[i + 1] : 0);
    res->hsh = a + b; if (res->left == 0 && res->right == 0
    && res->end == false) { delete(res); return 0; } else
    return res; } void walk_on_tree(Node *v, int x, int i =
    LG) { if (v == 0) return; if (v->end == true) cout <<
    x << endl; walk_on_tree(v->left, x, i - 1);
    walk_on_tree(v->right, x + (1 << i), i - 1); } ll get(
    Node *v1, Node *v2, int x, int i = LG) { if (v1 == 0 &&
    v2 == 0) return 0; if (v1 == 0) { if (v2->end) return
    x; if (v2->left) return get(0, v2->left, x, i - 1);
    return get(0, v2->right, x + (1 << i), i - 1); } if (v2
    == 0) { if (v1->end) return x; if (v1->left) return
    get(v1->left, 0, x, i - 1); return get(v1->right, 0, x
    + (1 << i), i - 1); } if (v1->end || v2->end) return x;
    if (v1->left == 0) { if (v2->left) return get(0, v2->
    left, x, i - 1); return get(v1->right ? v1->right : 0,
    v2->right ? v2->right : 0, x + (1 << i), i - 1); } if (
    v2->left == 0) { if (v1->left) return get(v1->left, 0,
    x, i - 1); return get(v1->right ? v1->right : 0, v2->
    right ? v2->right : 0, x + (1 << i), i - 1); } if (v1->
    left->hsh == v2->left->hsh) return get(v1->right ? v1->
    right : 0, v2->right ? v2->right : 0, x + (1 << i), i -
    1); return get(v1->left ? v1->left : 0, v2->left ? v2
    ->left : 0, x, i - 1); } void solve() { ll n; cin >> n;
    vector<ll>a(n), sig(n + 1); vector<pair<ll, ll>>arr(n)
    ; map<int, int>have; ll cnt = 1; for (ll i = 0; i < n;
    i++) { cin >> a[i]; arr[i] = {a[i], i}; } sort(arr.
    begin(), arr.end()); have[arr[0].first] = cnt; sig[cnt]
    = arr[0].first; for (ll i = 1; i < n; i++) { if (arr[i
    ].first == arr[i - 1].first) continue; cnt++; have[arr[
    i].first] = cnt; sig[cnt] = arr[i].first; } for (ll i =
    0; i < n; i++) { a[i] = have[a[i]]; } vector<Node *>tr
    (n + 1); tr[0] = new Node(); for (ll i = 1; i <= n; i
    ++) { if (find(tr[i - 1], a[i - 1])) tr[i] = del(tr[i -
    1], a[i - 1]); else tr[i] = upd(tr[i - 1], a[i - 1]);
    } ll q; cin >> q; ll res = 0; while (q--) { ll a, b;
    cin >> a >> b; ll l = a ^ res; ll r = b ^ res; res =
    get(tr[l - 1], tr[r], 0); res = sig[res]; cout << res
    << endl; } }
int main() { prec(); ll t = 1, test = 0; while (t--) { solve
    (); } return 0; }
```

6.8 SuffixArray

```
const int N = 3e5 + 9;
const int LG = 18;
void induced_sort(const vector<int> &vec, int val_range,
    vector<int> &SA, const vector<bool> &sl, const vector<
    int> &lms_idx) {
```

```
  vector<int> l(val_range, 0), r(val_range, 0);
  for (int c : vec) {
    if (c + 1 < val_range) ++l[c + 1]; ++r[c];
  }
  partial_sum(l.begin(), l.end(), l.begin());
  partial_sum(r.begin(), r.end(), r.begin());
  fill(SA.begin(), SA.end(), -1);
  for (int i = lms_idx.size() - 1; i >= 0; --i)
    SA[--r[vec[lms_idx[i]]]] = lms_idx[i];
  for (int i : SA)
    if (i >= 1 && sl[i - 1]) {
      SA[l[vec[i - 1]]++] = i - 1;
    }
  fill(r.begin(), r.end(), 0);
  for (int c : vec)
    ++r[c];
  partial_sum(r.begin(), r.end(), r.begin());
  for (int k = SA.size() - 1, i = SA[k]; k >= 1; --k, i = SA
      [k])
    if (i >= 1 && !sl[i - 1]) {
      SA[--r[vec[i - 1]]] = i - 1;
    }
}

vector<int> SA_IS(const vector<int> &vec, int val_range) {
  const int n = vec.size();
  vector<int> SA(n), lms_idx; vector<bool> sl(n); sl[n - 1]
      = false;
  for (int i = n - 2; i >= 0; --i) {
    sl[i] = (vec[i] > vec[i + 1] || (vec[i] == vec[i + 1] &&
        sl[i + 1]));
    if (sl[i] && !sl[i + 1]) lms_idx.push_back(i + 1);
  }
  reverse(lms_idx.begin(), lms_idx.end());
  induced_sort(vec, val_range, SA, sl, lms_idx);
  vector<int> new_lms_idx(lms_idx.size()), lms_vec(lms_idx.
      size());
  for (int i = 0, k = 0; i < n; ++i)
    if (!sl[SA[i]] && SA[i] >= 1 && sl[SA[i] - 1]) {
      new_lms_idx[k++] = SA[i];
    }
  int cur = 0;
  SA[n - 1] = cur;
  for (size_t k = 1; k < new_lms_idx.size(); ++k) {
    int i = new_lms_idx[k - 1], j = new_lms_idx[k];
    if (vec[i] != vec[j]) {
      SA[j] = ++cur; continue;
    }
    bool flag = false;
    for (int a = i + 1, b = j + 1;; ++a, ++b) {
      if (vec[a] != vec[b]) {
        flag = true; break;
      }
      if ((!sl[a] && sl[a - 1]) || (!sl[b] && sl[b - 1])) {
        flag = !((!sl[a] && sl[a - 1]) && (!sl[b] && sl[b -
            1])); break;
      }
    }
    SA[j] = (flag ? ++cur : cur);
  }
  for (size_t i = 0; i < lms_idx.size(); ++i)
    lms_vec[i] = SA[lms_idx[i]];
  if (cur + 1 < (int)lms_idx.size()) {
    auto lms_SA = SA_IS(lms_vec, cur + 1);
    for (size_t i = 0; i < lms_idx.size(); ++i) {
      new_lms_idx[i] = lms_idx[lms_SA[i]];
    }
  }
  induced_sort(vec, val_range, SA, sl, new_lms_idx);
  return SA;
}
vector<int> suffix_array(const string &s, const int LIM =
    128) {
  vector<int> vec(s.size() + 1);
  copy(begin(s), end(s), begin(vec)); vec.back() = '$';
  auto ret = SA_IS(vec, LIM); ret.erase(ret.begin());
  return ret;
}
struct SuffixArray {
  int n; string s; vector<int> sa, rank, lcp; vector<vector<
      int>> t; vector<int> lg;
  SuffixArray() {}
  SuffixArray(string _s) { n = _s.size(), s = _s, sa =
      suffix_array(s), rank.resize(n); for (int i = 0; i < n
      ; i++) rank[sa[i]] = i; costruct_lcp(), prec(), build
      (); }
  void costruct_lcp() { int k = 0; lcp.resize(n - 1, 0); for
       (int i = 0; i < n; i++) { if (rank[i] == n - 1) { k =
       0; continue; } int j = sa[rank[i] + 1]; while (i + k
      < n && j + k < n && s[i + k] == s[j + k]) k++; lcp[
      rank[i]] = k; if (k) k--; } }
  void prec() { lg.resize(n, 0); for (int i = 2; i < n; i++)
       lg[i] = lg[i / 2] + 1; }
  void build() { int sz = n - 1; t.resize(sz); for (int i =
      0; i < sz; i++) { t[i].resize(LG); t[i][0] = lcp[i]; }
       for (int k = 1; k < LG; ++k) for (int i = 0; i + (1
      << k) - 1 < sz; ++i) t[i][k] = min(t[i][k - 1], t[i +
      (1 << (k - 1))][k - 1]); }
  int query(int l, int r) { return min(t[l][lg[r - l + 1]],
      t[r - (1 << lg[r - l + 1]) + 1][lg[r - l + 1]]); }
  int get_lcp(int i, int j) { return (i == j) ? n - i :
      query(rank[i], rank[j] - 1); }
  int lower_bound(string &t) { int l = 0, r = n - 1, k = t.
      size(), ans = n; while (l <= r) { int mid = l + r >>
      1; if (s.substr(sa[mid], min(n - sa[mid], k)) >= t)
      ans = mid, r = mid - 1; else l = mid + 1; } return ans
      ; }
  int upper_bound(string &t) { int l = 0, r = n - 1, k = t.
      size(), ans = n; while (l <= r) { int mid = l + r >>
      1; if (s.substr(sa[mid], min(n - sa[mid], k)) > t) ans
       = mid, r = mid - 1; else l = mid + 1; } return ans; }
  // occurrences of s[p, ..., p + len - 1]
  pair<int, int> find_occurrence(int p, int len) { p = rank[
      p]; pair<int, int> ans = {p, p}; int l = 0, r = p - 1;
       while (l <= r) { int mid = l + r >> 1; if (query(mid,
       p - 1) >= len) ans.first = mid, r = mid - 1; else l =
       mid + 1; } l = p + 1, r = n - 1; while (l <= r) { int
       mid = l + r >> 1; if (query(p, mid - 1) >= len) ans.
      second = mid, l = mid + 1; else r = mid - 1; } return
      ans; }
};
```

6.9 SuffixAutomation

```
#include <bits/stdc++.h>
using namespace std;
const int N = 3e5 + 9;
// SuffixAutomaton: A structure to handle the Suffix
    Automaton
// len -> Largest string length of the corresponding endpos-
    equivalent class
// link -> Longest suffix that is another endpos-equivalent
    class
// firstpos -> 1-indexed end position of the first
    occurrence of the largest string of that node
// minlen(v) -> Smallest string of node v = len(link(v)) + 1
// terminal nodes -> Store the suffixes
struct SuffixAutomaton {
    struct node {
        int len, link, firstpos;
        map<char, int> nxt; // Transitions for each character
    };
    int sz, last;                // Size and last added state
    vector<node> t;              // Stores nodes of the
        automaton
    vector<int> terminal;        // Stores terminal states
    vector<long long> dp;        // Memoization for
        occurrences
```

```
    vector<vector<int>> g;        // Graph structure for
        linking nodes
    SuffixAutomaton() {}
    SuffixAutomaton(int n) { t.resize(2 * n); terminal.resize
        (2 * n, 0); dp.resize(2 * n, -1); sz = 1; last = 0;
        g.resize(2 * n); t[0].len = 0; t[0].link = -1; t[0].
        firstpos = 0; }
    void extend(char c) { int p = last; if (t[p].nxt.count(c)
        ) { int q = t[p].nxt[c]; if (t[q].len == t[p].len +
        1) { last = q; return; } int clone = sz++; t[clone]
        = t[q]; t[clone].len = t[p].len + 1; t[q].link =
        clone; last = clone; while (p != -1 && t[p].nxt[c]
        == q) t[p].nxt[c] = clone, p = t[p].link; return; }
        int cur = sz++; t[cur].len = t[last].len + 1; t[cur
        ].firstpos = t[cur].len; p = last; while (p != -1 &&
         !t[p].nxt.count(c)) t[p].nxt[c] = cur, p = t[p].
        link; if (p == -1) t[cur].link = 0; else { int q = t
        [p].nxt[c]; if (t[p].len + 1 == t[q].len) t[cur].
        link = q; else { int clone = sz++; t[clone] = t[q];
        t[clone].len = t[p].len + 1; while (p != -1 && t[p].
        nxt[c] == q) t[p].nxt[c] = clone, p = t[p].link; t[q
        ].link = t[cur].link = clone; } } last = cur; }
    void build_tree() { for (int i = 1; i < sz; i++) g[t[i].
        link].push_back(i); }
    void build(string &s) { for (auto x : s) extend(x),
        terminal[last] = 1; build_tree(); }
    long long cnt(int i) { //number of times i-th node occurs
         in the string
        if (dp[i] != -1) return dp[i];
        long long ret = terminal[i];
        for (auto &x : g[i]) ret += cnt(x);
        return dp[i] = ret;
    }
};
```

6.10 Trie

```
struct node { bool endmark; node* next[26 + 1] = {}; node()
    { endmark = false; } } *root = new node();
void insert(char* str, int len) { node* curr = root; for (
    int i = 0; i < len; i++) { int id = str[i] - 'a'; curr
    = curr->next[id] = curr->next[id] ? curr->next[id] :
    new node(); } curr->endmark = true; }
bool search(char* str, int len) { node* curr = root; for (
    int i = 0; i < len; i++) if (!(curr = curr->next[str[i]
     - 'a'])) return false; return curr->endmark; }
void del(node* cur) { for (int i = 0; i < 26; i++) if (cur->
    next[i]) del(cur->next[i]); delete(cur); }
int main()
{
 int num_word, query; char str[50];
 puts("ENTER NUMBER OF WORDS"), cin >> num_word;
 for (int i = 1; i <= num_word; i++) scanf("%s", str),
     insert(str, strlen(str));
 puts("ENTER NUMBER OF QUERY"), cin >> query;
 for (int i = 1; i <= query; i++) scanf("%s", str), puts(
     search(str, strlen(str)) ? "FOUND" : "NOT FOUND");
 del(root), return 0;
}
```

6.11 bitTrie

```
const int LG = 30;
struct Node { Node* child[2] = {}; };
struct trieBit {
    Node* root; trieBit() { root = new Node(); }
    void add(int x) { Node* cur = root; for (int i = LG; i >=
         0; i--) { if ((x >> i) & 1) { if (!cur->child[1])
        cur->child[1] = new Node(); cur = cur->child[1]; }
        else { if (!cur->child[0]) cur->child[0] = new Node
        (); cur = cur->child[0]; } } }
    Node* doDel(Node* v, int x, int i = LG) { if (i == -1) {
        delete(v); return 0; } if ((x >> i) & 1) v->child[1]
         = doDel(v->child[1], x, i - 1); else v->child[0] =
        doDel(v->child[0], x, i - 1); if (!v->child[0] && !v
        ->child[1] && i != LG) { delete(v); return 0; } else
         return v; }
    void del(int x) { root = doDel(root, x); }
    bool find(int x) { Node* cur = root; for (int i = LG; i
        >= 0; i--) { if ((x >> i) & 1) { if (!cur->child[1])
         return false; cur = cur->child[1]; } else { if (!
        cur->child[0]) return false; cur = cur->child[0]; }
        } return true; }
    int bestXorMatch(int x) { Node* cur = root; long long res
         = 0; for (int i = LG; i >= 0; i--) { if ((x >> i) &
         1) { if (cur->child[0]) cur = cur->child[0]; else {
         res += (1 << i); cur = cur->child[1]; } } else { if
         (cur->child[1]) { res += (1 << i); cur = cur->child
        [1]; } else cur = cur->child[0]; } } return res; }
};
```

6.12 kmp

```
// returns the longest proper prefix array of pattern p
// where lps[i]=longest proper prefix which is also suffix
    of p[0...i]
vector<int> build_lps(string p) {
 int sz = p.size();vector<int> lps;lps.assign(sz + 1, 0);
 int j = 0;lps[0] = 0;
 for (int i = 1; i < sz; i++) {
  while (j >= 0 && p[i] != p[j]) {
   if (j >= 1) j = lps[j - 1];
   else j = -1;
  }
  j++;lps[i] = j;
 }
 return lps;
}
vector<int>ans;
// returns matches in vector ans in 0-indexed
void kmp(vector<int> lps, string s, string p) {
 int psz = p.size(), sz = s.size();int j = 0;
 for (int i = 0; i < sz; i++) {
  while (j >= 0 && p[j] != s[i])
   if (j >= 1) j = lps[j - 1];
   else j = -1;
  j++;
  if (j == psz) {
   j = lps[j - 1];
   // pattern found in string s at position i-psz+1
   ans.push_back(i - psz + 1);
  }
  // after each loop we have j=longest common suffix of s
      [0..i] which is also prefix of p
 }
}
```

6.13 z algo

```
vector<int> z_function(string s) { int n = (int) s.length();
     vector<int> z(n); for (int i = 1, l = 0, r = 0; i < n;
     ++i) { if (i <= r) z[i] = min(r - i + 1, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
    if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1; } return
     z; }
```

7 Tree

7.1 Centriod Decomposition

```
struct centroid { vector<vector<int>> edges; vector<bool>
    vis; vector<int> par, sz; int n; void init(int s) { n =
```

```
    s; edges = vector<vector<int>>(n); vis = vector<bool>(
    n); par = vector<int>(n); sz = vector<int>(n); } void
    edge(int a, int b) { edges[a].pb(b); edges[b].pb(a); }
    int find_size(int v, int p = -1) { if (vis[v]) return
    0; sz[v] = 1; for (int x : edges[v]) { if (x != p) sz[v
    ] += find_size(x, v); } return sz[v]; } int
    find_centroid(int v, int p, int n) { for (int x : edges
    [v]) { if (x != p && !vis[x] && sz[x] > n / 2) return
    find_centroid(x, v, n); } return v; } void
    init_centroid(int v = 0, int p = -1) { find_size(v);
    int c = find_centroid(v, -1, sz[v]); vis[c] = true; par
    [c] = p; for (int x : edges[c]) if (!vis[x])
    init_centroid(x, c); } };
ll n, m, k, q, l, r, x, y, z; ll a[1000005], b[1000005], c
    [1000005]; string s, t; ll ans = 0; lca_lift lca;
    centroid ct; int best[100005]; void update(int v) {
    best[v] = 0; int u = v; while (ct.par[u] != -1) { u =
    ct.par[u]; best[u] = min(best[u], lca.get_dist(u, v));
    } } int query(int v) { int ans = best[v]; int u = v;
    while (ct.par[u] != -1) { u = ct.par[u]; ans = min(ans,
     best[u] + lca.get_dist(u, v)); } return ans; } void
    solve(int tc = 0) { cin >> n >> q; lca.init(n); ct.init
    (n); f0r(i, n) best[i] = 2e5; f0r(i, n - 1) { cin >> x
    >> y; --x; --y; lca.edge(x, y); ct.edge(x, y); } lca.
    init_lift(); ct.init_centroid(); update(0); f0r(i, q) {
     int t; cin >> t >> x; --x; if (t == 1) update(x); else
     cout << query(x) << '\n'; } }
```

7.2 Eular tour

```
vector<int>node[N];
int Intime[N], Outtime[N], Level[N] , a[N];
int timer = 1;
int n, q;
void EulerTour(int p, int par, int d){
    Intime[p] = timer++;
    Level[p] = d;
    for (auto i : node[p]){
        if (i == par)
            continue;
        EulerTour(i, p, Level[p] + 1);
    }
    Outtime[p] = timer;  //if timer++ then intime[u] to
         intime[v] can be find path query
}
struct BIT {
} T[2];
void Solve(){
    cin >> n >> q;
    for (int i = 1 ; i <= n ; i++)
        cin >> a[i];
    for (int i = 1 ; i < n ; i++){
        int u, v;
        cin >> u >> v;
        node[u].push_back(v);
        node[v].push_back(u);
    }
    EulerTour(1, 0, 0);
    for (int i = 1 ; i <= n ; i++){
        T[Level[i] % 2].upd(Intime[i], Intime[i], a[i]);
        T[!(Level[i] % 2)].upd(Intime[i], Intime[i], 0);
    }
    while (q--){
        int type;
        cin >> type;
        if (type == 1){
            ll x, val;
            cin >> x >> val;
            T[Level[x] % 2].upd(Intime[x], Outtime[x]-1, val)
                ;
            T[!(Level[x] % 2)].upd(Intime[x], Outtime[x]-1, -
                val);
        }
        else{
            ll x;
            cin >> x;
            cout << T[Level[x] % 2].query(Intime[x], Intime[x
                ]) << endl;
        }
    }
}
```

7.3 HLD(update on edge)

```
ll n, q; vector<ll>node[N]; ll a[N]; struct SegmentTree {
    vector<ll> tree, lazy, aa; SegmentTree() { tree.resize
    (4 * N); lazy.resize(4 * N); aa.resize(4 * N); } void
    build(ll node, ll b, ll e) { if (b == e) { tree[node] =
     0; lazy[node] = -1; return; } ll mid = (b + e) >> 1;
    build(2 * node, b, mid); build(2 * node + 1, mid + 1, e
    ); lazy[node] = -1; tree[node] = tree[2 * node] + tree
    [2 * node + 1]; } void push(ll node, ll b, ll e) { tree
    [node] = (e - b + 1) * lazy[node]; if (b != e) { lazy[2
     * node] = lazy[2 * node + 1] = lazy[node]; } lazy[node
    ] = -1; } void update(ll node, ll b, ll e, ll l, ll r,
    ll x) { if (lazy[node] != -1) push(node, b, e); if (l >
     e || r < b) return; if (l <= b && r >= e) { tree[node]
     = (e - b + 1) * x; if (b != e) { lazy[2 * node] = lazy
    [2 * node + 1] = x; } lazy[node] = -1; return; } ll mid
     = (b + e) >> 1; update(2 * node, b, mid, l, r, x);
    update(2 * node + 1, mid + 1, e, l, r, x); tree[node] =
     tree[2 * node] + tree[2 * node + 1]; } ll query(ll
    node, ll b, ll e, ll l, ll r) { if (lazy[node] != -1)
    push(node, b, e); if (l > e || r < b) return 0; if (l
    <= b && r >= e) { return tree[node]; } ll mid = (b + e)
     >> 1; return query(2 * node, b, mid, l, r) + query(2 *
     node + 1, mid + 1, e, l, r); } }; SegmentTree st; ll
    par[N][LG + 1], dep[N], sz[N]; void dfs(int u, int p =
    0) { par[u][0] = p; dep[u] = dep[p] + 1; sz[u] = 1; for
     (int i = 1; i <= LG; i++) { par[u][i] = par[par[u][i -
     1]][i - 1]; } for (auto v : node[u]) { if (v == p)
    continue; dfs(v, u); sz[u] += sz[v]; } } int lca(int u,
     int v) { } int intime[N], head[N]; int timer = 1; map<
    ll, ll>alledge[N]; void decompose(int p, int parent,
    int Head_node) { intime[p] = timer++; head[p] =
    Head_node; st.update(1, 1, n, intime[p], intime[p],
    alledge[parent][p]); int heavysize = -1, heavychild =
    -1; for (auto i : node[p]) { if (i != parent) { if (sz[
    i] > heavysize) heavysize = sz[i], heavychild = i; } }
    if (heavychild == -1) return; decompose(heavychild, p,
    Head_node); for (auto i : node[p]) { if (i ==
    heavychild || i == parent) continue; decompose(i, p, i)
    ; } } ll sumpath(int u, int v) { ll ans = 0; if (u == v
    ) return 0; while (head[u] != head[v]) { if (dep[head[u
    ]] > dep[head[v]]) swap(u, v); ans += st.query(1, 1, n,
     intime[head[v]], intime[v]); v = par[head[v]][0]; } if
     (dep[u] > dep[v]) swap(u, v); if (u != v) ans += st.
    query(1, 1, n, intime[u] + 1, intime[v]); return ans; }
     void reset(int n) { for (int i = 0; i <= n; i++) {
    intime[i] = head[i] = dep[i] = sz[i] = 0; a[i] = 0;
    node[i].clear(); alledge[i].clear(); } timer = 1; }
    void Solve() { cin >> n; st.build(1, 1, n); vector<pair
    <int, int>> edge; for (int i = 1; i < n; i++) { ll u, v
    , w; cin >> u >> v >> w; alledge[u][v] = w; alledge[v][
    u] = w; node[u].push_back(v); node[v].push_back(u);
    edge.push_back({u, v}); } dfs(1); decompose(1, 0, 1);
    int q; cin >> q; while (q--) { int type; cin >> type;
    if (type == 1) { ll id, x; cin >> id >> x; id--; int p
    = edge[id].f; int q = edge[id].s; if (dep[p] > dep[q])
    swap(p, q); st.update(1, 1, n, intime[q], intime[q], x)
    ; alledge[p][q] = x; alledge[q][p] = x; } else { int u,
     v; cin >> u >> v; int l = lca(u, v); if (u == v) {
    cout << 0 << endl; continue; } if (l == u || l == v) {
    if (dep[u] > dep[v]) swap(u, v); cout << sumpath(u, v)
    << endl; } else { cout << sumpath(l, u) + sumpath(l, v)
     << endl; } } } }
```

7.4 HLD(update on node)

```
int n, q; vector<int> node[N]; int a[N]; struct SegmentTree
    { vector<int> tree, lazy, aa; SegmentTree() { tree.
    resize(4 * N); lazy.resize(4 * N); aa.resize(4 * N); }
    void build(int node, int b, int e) { if (b == e) { tree
    [node] = 0; lazy[node] = -1; return; } int mid = (b + e
    ) >> 1; build(2 * node, b, mid); build(2 * node + 1,
    mid + 1, e); lazy[node] = -1; tree[node] = max(tree[2 *
     node], tree[2 * node + 1]); } void push(int node, int
    b, int e) { tree[node] = lazy[node]; if (b != e) { lazy
    [2 * node] = lazy[2 * node + 1] = lazy[node]; } lazy[
    node] = -1; } void update(int node, int b, int e, int l
    , int r, int x) { if (lazy[node] != -1) push(node, b, e
    ); if (l > e || r < b) return; if (l <= b && r >= e) {
    tree[node] = x; if (b != e) { lazy[2 * node] = lazy[2 *
     node + 1] = x; } lazy[node] = -1; return; } int mid =
    (b + e) >> 1; update(2 * node, b, mid, l, r, x); update
    (2 * node + 1, mid + 1, e, l, r, x); tree[node] = max(
    tree[2 * node], tree[2 * node + 1]); } int query(int
    node, int b, int e, int l, int r) { if (lazy[node] !=
    -1) push(node, b, e); if (l > e || r < b) return 0; if
    (l <= b && r >= e) { return tree[node]; } int mid = (b
    + e) >> 1; return max(query(2 * node, b, mid, l, r),
    query(2 * node + 1, mid + 1, e, l, r)); } };
    SegmentTree st; int par[N][LG + 1], dep[N], sz[N]; void
     dfs(int u, int p = 0) { par[u][0] = p; dep[u] = dep[p]
     + 1; sz[u] = 1; for (int i = 1; i <= LG; i++) { par[u
    ][i] = par[par[u][i - 1]][i - 1]; } for (auto v : node[
    u]) { if (v == p) continue; dfs(v, u); sz[u] += sz[v];
    } } int lca(int u, int v) { if (dep[u] < dep[v]) swap(u
    , v); for (int k = LG; k >= 0; k--) if (dep[par[u][k]]
    >= dep[v]) u = par[u][k]; if (u == v) return u; for (
    int k = LG; k >= 0; k--) if (par[u][k] != par[v][k]) u
    = par[u][k], v = par[v][k]; return par[u][0]; } int
    intime[N], head[N]; int timer = 1; void decompose(int p
    , int parent, int Head_node) { intime[p] = timer++;
    head[p] = Head_node; st.update(1, 1, n, intime[p],
    intime[p], a[p]); int heavysize = -1, heavychild = -1;
    for (auto i : node[p]) { if (i != parent) { if (sz[i] >
     heavysize) heavysize = sz[i], heavychild = i; } } if (
    heavychild == -1) return; decompose(heavychild, p,
    Head_node); for (auto i : node[p]) { if (i ==
    heavychild || i == parent) continue; decompose(i, p, i)
    ; } } int maxnode(int u, int v) { int ans = 0; while (
    head[u] != head[v]) { if (dep[head[u]] > dep[head[v]])
    swap(u, v); ans = max(ans, st.query(1, 1, n, intime[
    head[v]], intime[v])); v = par[head[v]][0]; } if (dep[u
    ] > dep[v]) swap(u, v); ans = max(ans, st.query(1, 1, n
    , intime[u], intime[v])); return ans; } void Solve() {
    cin >> n >> q; for (int i = 1; i <= n; i++) { cin >> a[
    i]; } st.build(1, 1, n); for (int i = 1; i < n; i++) {
    int u, v; cin >> u >> v; node[u].push_back(v); node[v].
    push_back(u); } dfs(1); decompose(1, 0, 1); while (q--)
     { int type; cin >> type; if (type == 1) { int u, x;
    cin >> u >> x; st.update(1, 1, n, intime[u], intime[u],
     x); } else { int u, v; cin >> u >> v; int l = lca(u, v
    ); cout << max(maxnode(l, u), maxnode(l, v)) << " "; }
    } }
```

7.5 LCA

```
int n, maxHeight, timer;
vector<vector<int>> adj, up;
vector<int> tin, tout, sts, dep, lg(sz + 1);
void dfs(int cur, int par) {
    tin[cur] = ++timer; up[cur][0] = par; dep[cur] = dep[par]
         + 1;
    for (int i = 1; i <= maxHeight; ++i)
        up[cur][i] = up[up[cur][i - 1]][i - 1];
    for (int child : adj[cur]) {
        if (child != par) {
            dfs(child, cur); sts[cur] += sts[child];
        }
    }
    sts[cur]++; tout[cur] = ++timer;
}
bool is_ancestor(int u, int v) {
    return tin[u] <= tin[v] && tout[u] >= tout[v];
}
int lca(int u, int v) {
    if (is_ancestor(u, v))
        return u;
    if (is_ancestor(v, u))
        return v;
    for (int i = maxHeight; i >= 0; --i) {
        if (!is_ancestor(up[u][i], v))
            u = up[u][i];
    }
    return up[u][0];
}
void toBinary(int n, vector<bool> &v) {
    while (n) {
        v.pb(n % 2); n /= 2;
    }
}
ll lift(int u, int i) {
    vector<bool> v; toBinary(i, v);
    for (int i = 0; i < v.size(); i++) {
        if (v[i]) u = up[u][i];
    }
    return u;
}
int kth(int u, int k) {
    return lift(u, k);
}
// kth node on the path from u to v, where 0th node is u
int go(int u, int v, int k) {
    int l = lca(u, v); int d = dep[u] + dep[v] - (dep[l] <<
         1);
    assert(k <= d);
    if (dep[l] + k <= dep[u])
        return kth(u, k);
    k -= dep[u] - dep[l];
    return kth(v, dep[v] - dep[l] - k);
}
void createlogTable() {
    lg[1] = 0;
    for (int i = 2; i <= sz; i++)
        lg[i] = lg[i / 2] + 1;
}
void preprocess(int root) {
    tin.assign(n, 0); tout.assign(n, 0); sts.assign(n, 0);
         dep.assign(n, 0);
    timer = 0; maxHeight = lg[n];
    up.assign(n, vector<int>(maxHeight + 1)); dfs(root, root)
         ;
}
int dist(int u, int v) {
    int l = lca(u, v); return dep[u] + dep[v] - 2 * dep[l];
}
createlogTable();
preprocess(0);
```